

DIGITALIZATION OF HOSTEL MANAGEMENT SYSTEM

Project report submitted at the end of 8th semester in the partial fulfilment of the requirements for the award of degree of

BACHELOR OF TECHNOLOGY in INFORMATION TECHNOLOGY

By

NAMALA EEEKSHITHA	322107311030
PENTAPATI CHANDINI	322107311035
TEDLAPU HARSHITHA SAI LAKSHMI	322107311050

Under the guidance of

Prof. KUNJAM NAGESWARA RAO



DEPARTMENT OF INFORMATION TECHNOLOGY & COMPUTER APPLICATIONS

A.U COLLEGE OF ENGINEERING(A),

ANDHRA UNIVERSITY,

VISAKHAPATANAM

2022 – 2026

DEPARTMENT OF INFORMATION TECHNOLOGY & COMPUTER APPLICATIONS

A.U COLLEGE OF ENGINEERING(A),

ANDHRA UNIVERSITY,

VISAKHAPATANAM



CERTIFICATE

This is to certify that the project report entitled **DIGITALIZATION OF HOSTEL MANAGEMENT SYSTEM** is a bonafide record of work carried out by Namala Eekshitha, Pentapati Chandini ,Tedlapu Harshitha Sai Lakshmi with Registered Numbers: 322107311035, 322107311030, 322107311050 under the supervision and guidance of **Prof. Kunjam Nageswara Rao**. This project is submitted to the Department of Information Technology and Computer Applications, Andhra University College of Engineering in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology from Andhra University for the academic year 2025-2026. To the best of my knowledge, the results embodied in this report have not been submitted to any other University or Institute for the award of any degree or diploma.

Prof. Kunjam Nageswara Rao
Project Guide

Prof. D. Lalitha Bhaskari
Head of the Department

ACKNOWLEDGEMENT

I would like to show my greatest appreciation To Prof. Kunjam Nageswara Rao enough for his tremendous support and help without his encouragement and guidance this project would not have materialized.

I would like to thank Prof. D. Lalitha Bhaskari, Board of Studies Chairman and Head of the Department, Information Technology and Computer Applications, Andhra University College of Engineering(A), for her encouragement and valuable guidelines in bringing shape to the dissertation.

I wish to express thanks to Prof. M. Shashi, Principal, Andhra University College of Engineering (A), for helping me in completing the project work on time.

I would like to thank teaching staff and non-teaching staff members of the department of Information Technology and Computer Applications, Andhra University College of Engineering (A), Visakhapatnam, for their constant support in successful completion of my study.

Finally, I express my indebtedness to my beloved parents and friends without whose blessings and encouragement I would not have completed my work fruitfully.

NAMALA EEKSHITHA
322107311030
PENTAPATI CHANDINI
322107311035
TEDLAPU HARSHITHA SAI LAKSHMI
22107311050
B.Tech (Information Technology)

DECLARATION

I declare that the project report entitled **DIGITALIZATION OF HOSTEL MANAGEMENT SYSTEM** has been done by us in partial fulfillment of requirement for the award of degree of Bachelor of Technology in Information Technology, during the academic year 2022-2026 under the guidance of Prof. Kunjam Nageswara Rao, department of Information Technology and Computer Applications, AU College of Engineering(A), Andhra University, Visakhapatnam. I, here by declare that this project work has not been submitted to any other universities/institutions for the award of any degree.

PLACE: VISAKHAPATNAM

Date:

NAMALA EEKSHITHA

(321107311030)

PENTAPATI CHANDINI

(322107311035)

TEDLAPU HARSHITHA

SAI LAKSHMI

(322107311050)

ABSTRACT

ABSTRACT

Digitalization of Hostel Management System digitalizes and streamlines hostel administration by providing a unified platform for students, administrators and organization to manage registration, room allocation, fee payments, and certificate generation, eliminating manual paperwork and ensuring transparent, efficient hostel operations. The main aim is to digitalize hostel management operations by providing an automated, centralized platform for seamless student and administrative interactions.

This system is designed around a structured, role-based workflow where users provide inputs according to their access level. These inputs are systematically validated, organized, and stored in a centralized repository to ensure data accuracy, consistency, and integrity. All data elements, including student details, registrations, billing details, certificates, and hostel block allocations, are interconnected and maintained within a unified structure, enabling seamless coordination across different modules. As the workflow progresses, the system dynamically processes and maps inputs to available resources, supporting efficient room allocation, continuous updates, and real-time synchronization of records while eliminating redundancy. The system also manages financial inputs by organizing and processing them into structured outputs such as billing records and transaction details, ensuring traceability and accurate tracking. Administrative operations include monitoring, retrieval, and management of data, along with bulk data handling through structured import mechanisms, allowing efficient processing of large datasets such as student records. Additionally, the system generates outputs in the form of confirmations, summaries, analytical views, and official documents like certificates, ensuring clarity and accessibility. The overall workflow follows a continuous cycle of input collection, validation, processing, synchronization, and output generation, maintaining consistency across all modules. It supports coordinated operations and ensures that updates in one component are reflected throughout the system. By providing a centralized and unified platform, the system minimizes manual effort, reduces data duplication, and enhances operational efficiency. This approach results in a scalable, reliable, and transparent solution that improves the overall management and user experience of hostel administration. Backup and recovery strategies are implemented to safeguard data against loss or system failures, while optimized database queries and caching techniques improve overall system performance and responsiveness. The architecture is designed to be modular and extensible, allowing easy integration of new features such as mobile access, automated notifications, and advanced analytics. Furthermore, the system supports audit logs and activity tracking, enabling administrators to monitor actions and maintain accountability. These enhancements strengthen the robustness of the system and ensure it can efficiently handle growing user demands while maintaining high reliability and security standards.

The system is designed and developed using React.js, Django, Django REST Framework is used to build and expose RESTful APIs that facilitate smooth communication between the frontend and backend, PostgreSQL, Postman, Git, Visual Studio Code as the primary coding environment and implemented under Windows Operating System.

INDEX

	PAGE NO.
CHAPTER 1 INTRODUCTION	
1.1 Overview	2
1.2 Computational Approach	3
1.3 Existing System	4
1.4 Problem Statement	5
1.5 Proposed System	6
CHAPTER 2 LITERATURE SURVEY	
2.1 Introduction	8
CHAPTER 3 SYSTEM ANALYSIS AND DESIGN	
3.1 Introduction	11
3.2 System Requirements	12
3.3 Design Approaches	13
3.4 UML Diagrams	17
3.5 Methodology	28
3.6 Algorithm	29
CHAPTER 4 IMPLEMENTATION	
4.1 Introduction to Technologies	33
4.2 Sample Code	35
CHAPTER 5 TESTING	
5.1 Introduction	42
5.2 Test Cases	44
CHAPTER 6 RESULTS AND ANALYSIS	48
CHAPTER 7 CONCLUSION AND FUTURE SCOPE	63
CHAPTER 8 REFERENCES	65

LIST OF FIGURES

Figure No.	Name of the Figure	Page No
3.4.1	Class Diagram	19
3.4.2	Sequence Diagram	22
3.4.3	Use Case Diagram	24
3.4.4	Activity Diagram	26
6.1	Home page	48
6.1.1	Registration page	49
6.1.2	Login page	49
6.1.3	Student profile page	50
6.1.4	Student Dashboard	51
6.1.5	Room Allocation page	51
6.1.6	Room booking Confirmation	52
6.1.7	Fee payment	53
6.2.1	Admin Login	53
6.2.2	Admin Dashboard	54
6.2.3	Student Records search registry	55
6.2.4	Billing details	55
6.2.5	Registrations	56
6.2.6	Certificates	57
6.2.6.1	Resident Certificate	57
6.2.6.2	No Dues Certificate	58
6.2.6.3	Fee Estimation Certificate	58
6.2.7	Upload Excel data	59
6.2.8	Hostel allocation	59
6.2.9	Know Your Details	60
6.2.9.1	Mess fee payment	61

INTRODUCTION

INTRODUCTION

1.1 OVERVIEW

In recent years, managing hostel operations in educational institutions has become increasingly complex due to the growing number of students, multiple hostel blocks, and the need for efficient resource allocation. Traditionally, hostel management relies on manual processes such as maintaining physical registers, handling paperwork for student registration, room allotment, and fee collection. These methods are time-consuming, prone to human error, and difficult to manage at scale, often leading to data inconsistencies, delays in allocation, and lack of transparency. With the increasing need for efficiency and accuracy, there is a strong demand to digitize and automate hostel operations.

The Digitalization of Hostel Management System is a web-based application developed to address these challenges by providing a centralized platform for managing hostel-related activities. The system is designed to serve multiple users, including students, administrators, and organizations, through a role-based access mechanism. Each user can securely log in and access functionalities specific to their responsibilities, ensuring proper authorization and controlled access to data. Students can register, manage their profiles, apply for hostel rooms, and make fee payments, while administrators and organizations can monitor student records, manage allocations, and oversee overall hostel operations.

The need for such a system arises from the inefficiencies of traditional hostel management, where manual record-keeping often results in misplaced data, delayed processes, and difficulty in tracking student and payment information. Managing large volumes of student data and allocating rooms based on availability becomes a repetitive and error-prone task. Digitalization of Hostel Management provides a structured digital solution to streamline these processes by enabling automated workflows, centralized data storage, and real-time updates, thereby reducing administrative burden and improving operational efficiency.

The system also includes an online payment mechanism that allows students to pay hostel and mess fees through multiple convenient options. Payment confirmations are generated instantly, and receipts are provided digitally, ensuring transparency and proper record maintenance. Additionally, administrators are provided with dashboards that display registered student counts, enabling them to allocate hostel blocks efficiently based on occupancy and availability, ensuring balanced utilization of resources.

The primary goals of Digitalization of Hostel Management include:

- To provide a user-friendly interface for managing hostel operations efficiently.
- To automate room allocation and reduce manual errors in record keeping.
- To enable secure and convenient fee payment with proper tracking.
- To ensure controlled access through role-based functionality.
- To reduce time and effort involved in hostel administration processes.

This project represents a step toward modernizing hostel management by introducing a structured and centralized system. By incorporating features such as automated workflows, secure access, digital payments, and administrative monitoring, the system enhances efficiency, improves transparency, and provides a streamlined solution for managing hostel facilities in educational institutions.

1.2 COMPUTATIONAL APPROACH

The system is developed using a structured and modular computational approach that ensures efficiency, scalability, and reliability in handling hostel operations. The system follows a layered architecture that separates user interaction, core processing, and data management, enabling smooth communication between different components. This structured design reduces system complexity, improves maintainability, and allows each module to function independently while contributing to the overall workflow.

The system operates on a client-server model where user requests are processed through a well-defined sequence of operations. The user interface provides an intuitive environment where users can perform activities such as registration, login, profile management, room selection, and fee payment. Inputs from users are captured and forwarded to the processing layer, where validations and logical operations are performed. Based on the processed results, the system generates appropriate responses and dynamically updates the interface to ensure a seamless user experience.

The core processing layer is responsible for implementing the business logic and managing all system functionalities. It handles user authentication and role-based access control to ensure that students, administrators, and organizations can only access authorized features. This layer manages student records, registrations, billing details, and certificate generation, ensuring that all data is processed accurately and efficiently. It also supports hostel block allocation by analysing the number of registered students and distributing them across available blocks to achieve balanced occupancy. Additionally, the system includes bulk data handling through Excel upload functionality, allowing administrators to insert and manage large volumes of student data efficiently. All these operations are governed by validation rules to maintain consistency and prevent errors.

The data management layer maintains structured storage of all essential information, including student details, billing records, registration data, certificates, and hostel allocation information. Efficient data handling techniques ensure quick retrieval, updates, and secure storage of records. This organized approach minimizes redundancy and enhances data integrity, making the system reliable for large-scale usage.

The system also incorporates external functionalities to enhance performance and usability. Online payment processing enables users to complete transactions securely, while automated notification mechanisms provide confirmations and important updates. These features improve transparency and ensure effective communication between users and the system.

The overall computational workflow of the system can be summarized as follows:

- User interacts with the system through the interface
- Requests are sent to the processing layer
- The system performs authentication and applies business rules
- Data such as student details, billing details, registrations, certificates, and hostel allocations are processed and managed
- Bulk data operations are handled through Excel upload where required
- External services are triggered when necessary (such as payments or notifications)
- The system generates a response and updates the interface accordingly

The development of this system follows a modular and iterative approach, allowing continuous improvements based on testing and user requirements. By structuring the system into well-defined components and ensuring smooth interaction between them, the Digitalization of Hostel Management provides a reliable, efficient, and scalable solution that overcomes the limitations of traditional manual hostel administration.

1.3 EXISTING SYSTEM

The traditional hostel management system followed prior to this development was entirely manual, relying on physical paperwork, in-person interactions, and basic spreadsheet tools for every aspect of hostel administration. The student registration process required every student to physically visit the hostel office during working hours, collect printed application forms, fill them in by hand, attach physical copies of documents including Aadhaar card, photographs, birth certificate, and caste certificate, and submit everything at the office counter. Staff manually verified each document, checked for completeness, and filed the forms in physical registers. Any errors or missing documents required the student to return on another day, causing repeated delays and inconvenience.

The Room allocation was handled manually by wardens based on information from physical occupancy registers maintained at the hostel office. There was no real-time visibility into which rooms were occupied and which were available, frequently resulting in errors such as the same room being allocated to two different students. Students had no mechanism to view available rooms, check block-wise availability, or express preferences — all decisions were made entirely at the discretion of staff without any transparent or systematic process, making the allocation susceptible to human error and favouritism.

Fee collection was conducted entirely through cash payments or demand drafts submitted at the hostel office counter, with students standing in long queues during designated collection periods. Manual paper receipts were issued for each payment, which students had to carefully preserve as proof of transaction. There was no digital payment record, making it extremely difficult to resolve disputes or provide payment proof for scholarship or loan applications. Mess fee calculation was done manually by staff based on attendance registers, a process that was slow, inconsistent, and prone to arithmetic errors.

The Record maintenance was done through physical registers, files, and folders at the hostel office. Searching for a specific student's records required staff to manually go through multiple registers, which was extremely time-consuming when handling thousands of students. Data was duplicated across multiple registers, with inconsistencies arising whenever updates were not applied uniformly. Physical records were also vulnerable to damage from fire, flooding, or misplacement, with no digital backup available to recover lost data.

Certificate issuance was one of the most cumbersome processes in the existing system. Students requiring residence certificates, no dues certificates, or loan estimation slips had to submit written applications at the hostel office and wait for several days while staff manually prepared the documents. There was no standardized template, leading to inconsistencies in the format and content of certificates issued by different staff members. Verifying whether a student had pending dues before issuing a no dues certificate required manually checking through payment registers, which was unreliable and time-consuming. The absence of any digital communication channel meant students had no way to receive updates about fee deadlines, room allocation schedules, or other hostel announcements except through physical notice boards, often resulting in missed deadlines and administrative complications.

Drawbacks of the Existing System

- The manual nature of the previous administration resulted in the following critical issues:
- **Administrative Inefficiency:** The heavy reliance on physical presence and manual data entry led to significant delays for both students and staff.
- **Lack of Real-Time Data:** There was no visibility into room availability, leading to errors such as double-allocations and logistical confusion.
- **Financial Risk and Lack of Transparency:** Cash-only transactions without digital trails made dispute resolution difficult and complicated the process for students applying for scholarships or loans.
- **Human Error:** Manual calculations of mess fees and manual verification of "No Dues" were prone to arithmetic mistakes and oversight.
- **Data Vulnerability:** Physical records lacked backups and were highly susceptible to loss, misplacement, or damage from environmental factors like fire or flooding.
- **Communication Gaps:** Dependence on physical notice boards meant students often missed critical updates regarding fee deadlines or administrative schedules.
- **Inconsistent Standardization:** The lack of digital templates led to inconsistencies in the format and content of official certificates and records.

1.4 PROBLEM STATEMENT

The current hostel administration relies heavily on manual processes, resulting in inefficiencies, errors, and lack of transparency. There is no centralized digital platform to manage key operations such as student details, registrations, billing details, certificate generation, Excel-based bulk data handling, and hostel block allocations. Room allocation is

unorganized due to the absence of real-time availability tracking, while fee collection is handled manually without proper digital records, leading to accountability issues. Administrative tasks like record retrieval and certificate generation are time-consuming and increase workload. Additionally, students must physically visit the hostel office for every service, causing delays and inconvenience. These limitations lead to data duplication, poor resource utilization, and an overall inefficient system, highlighting the need for a unified digital solution to automate and streamline hostel management processes.

1.5 PROPOSED SYSTEM

The proposed system is a web-based application named Digitalization of Hostel Management System, designed to automate and streamline hostel administration processes in an organized and efficient manner. It provides a centralized platform where administrators and wardens can manage student records, accommodation details, and financial transactions through structured, role-based workflows. This system replaces traditional manual methods that are time-consuming, error-prone, and difficult to maintain, ensuring improved accuracy and operational efficiency.

The application allows authorized users to access the system through secure login credentials and perform operations based on their roles. Students can register online, manage their profiles by submitting personal, academic, communication, and parental details, and upload required documents such as identification proofs and photographs. The system also enables real-time room allocation based on predefined eligibility criteria such as year, block, and availability, ensuring optimal utilization of hostel resources. Additionally, students can access a mess fee portal to view and complete payments, with support for multiple payment modes and instant confirmation through downloadable receipts and notifications.

For wardens and administrators, the system offers a comprehensive dashboard to oversee the entire hostel workflow. They can quickly search and access detailed student records using unique identifiers, monitor billing information including mess fees and electricity charges with accurate timestamps, and ensure transparency in all financial transactions. The system also supports automated document generation, including residence certificates, no-dues certificates with pending balance alerts, and loan estimation slips, reducing manual paperwork and processing time. Furthermore, bulk data upload functionality allows administrators to efficiently add large volumes of student data, minimizing repetitive manual entry.

As a web-based platform, the system is accessible from any internet-enabled device, providing flexibility and ease of use. Its intuitive interface ensures that users can navigate and perform tasks without requiring advanced technical knowledge. By digitizing hostel operations, the proposed system significantly reduces administrative workload, eliminates data redundancy, enhances record accuracy, and ensures a smooth, reliable, and transparent hostel management process.

LITERATURE SURVEY

LITERATURE SURVEY

Sharma and others developed a hostel management system using PHP and MySQL, where the primary focus was on student registration, room allocation, and basic fee management. Their system provided a simple web interface for administrators to manage hostel records. However, it lacked a modular design and advanced features such as real-time updates and secure payment integration. The system followed a tightly coupled architecture, which made it difficult to scale or extend with new functionalities. Additionally, the absence of automation increased dependency on manual intervention for several administrative tasks.

Reddy and Kumar implemented a web-based hostel system using HTML, CSS, JavaScript, and MySQL, enabling students to apply for rooms and administrators to manage allocations. While this system improved accessibility compared to manual methods, it lacked automation in billing processes and did not include any online payment mechanism. The system required manual verification and updates, which increased processing time. Furthermore, the user interface was static and did not provide dynamic interaction or real-time data updates.

Gupta and others designed a hostel management system using PHP with Bootstrap and MySQL, focusing on improving the user interface. Their system enhanced visual presentation and usability to some extent. However, it still followed a monolithic architecture without clear separation between frontend and backend components. This made maintenance complex and limited the ability to integrate advanced features. The system also lacked scalability and could not efficiently handle large datasets or concurrent users.

Choudhary and others developed a system using the Laravel framework (PHP) and MySQL, which introduced better code organization compared to traditional PHP applications. Their system improved development structure and maintainability. However, it did not integrate modern frontend frameworks, resulting in limited interactivity. Additionally, the system lacked features such as real-time notifications, online payment integration, and automated workflows, which are essential for modern applications.

Patel and others proposed a lightweight hostel management system using Python Flask and SQLite, which supported basic CRUD operations and user authentication. The system was simple and easy to deploy, making it suitable for small-scale applications. However, it was not designed to handle large datasets or multiple concurrent users. The lack of advanced features such as automated allocation, payment integration, and role-based access limited its usability in real-world institutional environments.

Verma and others implemented a backend-focused system using Django and SQLite, which improved data handling and security compared to PHP-based systems. The use of Django provided better structure and reliability. However, the system lacked a modern frontend framework, resulting in a less interactive user experience. It also did not include integration with third-party services like payment gateways or email notifications, reducing its overall functionality.

Rao and others developed a system using Angular, Node.js, and MongoDB, which offered improved frontend responsiveness and dynamic interaction. The system provided a better user experience compared to traditional applications. However, it primarily focused on frontend design and lacked comprehensive backend automation. Features such as billing logic,

certificate generation, and structured data handling were either limited or absent.

Nair and others studied web-based hostel management platforms and emphasized the importance of centralized control and role-based access. Their work highlighted improvements in usability and data management across different user roles. However, the system did not fully implement automation in processes such as room allocation and fee calculation. It also lacked integration with external services like online payment systems and real-time notification mechanisms.

SYSTEM ANALYSIS AND DESIGN

SYSTEM ANALYSIS AND DESIGN

3.1 INTRODUCTION

Digitalization of Hostel Management System Analysis is a crucial phase in the Software Development Life Cycle (SDLC) that focuses on identifying existing issues and designing an effective solution. In the Digitalization of Hostel Management System, this phase analyses the limitations of traditional hostel management methods, such as maintaining physical records, inefficient hostel block and floor allocation, and lack of transparency in billing and registrations. These manual processes are time-consuming, prone to errors, and create difficulties for both students and administrative staff.

The analysis begins by understanding the requirements of key stakeholders, including students, wardens, and administrators. Students need a simple and efficient platform to manage their profiles, upload documents, complete registrations, and make fee payments. Wardens and administrators require a centralized system to manage student records, billing details, registrations, certificates, Excel-based data uploads, and hostel block and floor allocations. These requirements help define the system's functionalities and constraints, such as room availability, eligibility criteria, allocation rules, and secure data handling.

During this phase, the system structure and data flow are carefully examined. The system includes interconnected modules such as authentication, profile management, student records, billing management, registrations, certificate handling, Excel upload, and hostel allocation. All modules are integrated with a centralized database to maintain consistency, ensure data integrity, and enable real-time updates. Workflow diagrams and database schemas are used to visualize how different components interact, helping in designing a scalable and user-friendly system.

The primary objective of this system is to automate and streamline hostel operations using predefined rules and structured workflows. It minimizes manual intervention, reduces errors, and enhances efficiency, transparency, and accessibility. The system is also designed to support future enhancements such as mobile application integration, advanced analytics, IoT-based monitoring, and scalability to accommodate additional hostel facilities. A feasibility study is also conducted to evaluate the technical and operational viability of the proposed system.

3.2 SYSTEM REQUIREMENTS

System Requirements Specification (SRS) :

A System Requirements Specification (SRS) is a formal document that defines the expected functionalities and constraints of a software system. It acts as a clear agreement between developers and users, ensuring that all requirements are well understood and documented before development begins. In the Digitalization of Hostel Management System, the SRS focuses on

streamlining hostel operations such as student records management, registrations, billing, and hostel allocation.

The SRS includes both functional and non-functional requirements, describing how the system behaves under various conditions and how users interact with it. It provides a clear understanding of system capabilities, limitations, and expected performance, forming the foundation for system design and implementation.

3.2.1 Functional Requirements

The functional requirements describe the operations and services that the system must perform. These define how the system responds to user inputs and interactions across different modules.

- The system should allow students to register by providing personal, academic, and contact details.
- The system should provide secure login access for students, wardens, and administrators.
- The system should allow users to reset their passwords through a verification process.
- The system should enable students to complete and update their profiles, including personal, academic, and parent/guardian information.
- The system should support uploading of required documents such as identification proofs and photographs.
- The system should allow administrators to manage and maintain student records efficiently.
- The system should support hostel block and floor allocation based on predefined rules and eligibility criteria.
- The system should display available rooms according to allocation settings and constraints.
- The system should allow students to select and confirm room booking.
- The system should provide detailed billing information, including mess fees, duration, and total amount.
- The system should support secure online payment processing.
- The system should generate payment receipts and provide confirmation notifications.
- The system should maintain billing records, including payment status and transaction history.
- The system should support certificate generation such as residence and no-dues certificates.
- The system should allow bulk upload of student data for efficient data handling.

3.2.2 Non-Functional Requirements

Non-functional requirements define the quality attributes and performance standards of the system.

- **Usability:** The system should provide a simple, user-friendly, and intuitive interface for easy navigation.
- **Performance:** The system should efficiently handle multiple users and operations without noticeable delays.
- **Security:** Access should be restricted to authorized users, and sensitive data must be securely protected.
- **Reliability:** The system should function consistently without failures or data loss.
- **Scalability:** The system should support an increase in users, data, and hostel facilities without performance degradation.
- **Maintainability:** The system should be structured in a way that allows easy updates, modifications, and debugging.

3.2.3 Hardware Requirements

- Processor: Basic computing processor or higher
- RAM: Minimum 4 GB
- Storage: Sufficient space for storing system data and records
- Internet Connection: Required for accessing the system

The system can be accessed using desktops, laptops, or mobile devices with internet connectivity.

3.2.4 Software Requirements

- A compatible operating environment capable of running web-based applications
- A web browser for accessing the system interface
- Basic development and testing tools for system maintenance and updates
- Version control support for managing system changes

The system is designed as a web-based application that can run on standard platforms without dependency on specific technologies, ensuring flexibility and adaptability.

3.3 DESIGN APPROACHES

The design approach for the Digitalization of Hostel Management System focuses on automation, modularity, and user-centric functionality. The system transforms traditional manual hostel operations into a structured digital workflow, ensuring that processes such as

student registration, profile management, hostel allocation, billing, certificate generation, and record maintenance are handled efficiently and accurately. Each module is designed to function independently while maintaining seamless integration with other modules, enabling a well-organized and flexible system.

The system follows a layered design approach that separates user interaction, system processing, and data management. This separation reduces complexity and improves maintainability. The user interaction layer ensures smooth navigation and accessibility for students, wardens, and administrators. The processing layer manages core operations such as validation, allocation logic, billing calculations, and record updates. The data layer securely stores and manages all system information in a structured format, ensuring consistency and integrity.

A key aspect of the design is structured communication between system components, enabling smooth data exchange and flexibility. This allows the system to support future enhancements such as mobile access or integration with additional services without major structural changes. The frontend is designed using reusable components such as forms, dashboards, and navigation sections, reducing redundancy and improving consistency. The backend follows a structured approach to separate data handling, business logic, and request processing, ensuring clean and maintainable system behaviour.

Automation plays a significant role in the system design. Hostel allocation is managed based on predefined conditions such as student year, hostel block, and floor availability, ensuring fairness and proper distribution. Similarly, billing calculations, payment tracking, and certificate generation are automated, reducing manual effort and minimizing errors. Validation mechanisms are implemented throughout the system to ensure data accuracy and prevent invalid inputs.

Security is ensured through role-based access control, where students, wardens, and administrators have different levels of access and permissions. This protects sensitive data and restricts actions based on user roles. The system is also designed to be scalable, allowing the addition of new features such as analytics, reporting tools, and expansion to additional hostel facilities without affecting existing functionalities.

3.3.1 System Architecture

The system architecture is designed using a multi-layered client-server model that ensures efficient communication, processing, and data management. It consists of three main layers: user interaction layer, processing layer, and data layer.

The user interaction layer provides an interface where users can perform actions such as registration, login, profile management, hostel allocation, and payment processing. It is designed to be interactive and responsive, ensuring that users receive real-time updates, such as room availability during selection.

The processing layer acts as the core of the system, handling all operations such as authentication, authorization, validation, hostel allocation logic, billing calculations, and

operations into a structured digital workflow, ensuring that processes such as student registration, profile management, hostel allocation, billing, certificate generation, and record maintenance are handled efficiently and accurately. Each module is designed to function independently while maintaining seamless integration with other modules, enabling a well-organized and flexible system.

The system follows a layered design approach that separates user interaction, system processing, and data management. This separation reduces complexity and improves maintainability. The user interaction layer ensures smooth navigation and accessibility for students, wardens, and administrators. The processing layer manages core operations such as validation, allocation logic, billing calculations, and record updates. The data layer securely stores and manages all system information in a structured format, ensuring consistency and integrity.

A key aspect of the design is structured communication between system components, enabling smooth data exchange and flexibility. This allows the system to support future enhancements such as mobile access or integration with additional services without major structural changes. The frontend is designed using reusable components such as forms, dashboards, and navigation sections, reducing redundancy and improving consistency. The backend follows a structured approach to separate data handling, business logic, and request processing, ensuring clean and maintainable system behaviour.

Automation plays a significant role in the system design. Hostel allocation is managed based on predefined conditions such as student year, hostel block, and floor availability, ensuring fairness and proper distribution. Similarly, billing calculations, payment tracking, and certificate generation are automated, reducing manual effort and minimizing errors. Validation mechanisms are implemented throughout the system to ensure data accuracy and prevent invalid inputs.

Security is ensured through role-based access control, where students, wardens, and administrators have different levels of access and permissions. This protects sensitive data and restricts actions based on user roles. The system is also designed to be scalable, allowing the addition of new features such as analytics, reporting tools, and expansion to additional hostel facilities without affecting existing functionalities.

3.3.1 System Architecture

The system architecture is designed using a multi-layered client-server model that ensures efficient communication, processing, and data management. It consists of three main layers: user interaction layer, processing layer, and data layer.

The user interaction layer provides an interface where users can perform actions such as registration, login, profile management, hostel allocation, and payment processing. It is designed to be interactive and responsive, ensuring that users receive real-time updates, such as room availability during selection.

The processing layer acts as the core of the system, handling all operations such as authentication, authorization, validation, hostel allocation logic, billing calculations, and

request handling. When a user performs an action, the system processes the request, applies necessary rules, interacts with stored data, and returns the appropriate response.

The data layer stores all system information in a structured manner, including student details, room information, allocations, billing records, and certificates. It ensures data consistency and integrity by maintaining relationships between different entities, allowing accurate tracking and retrieval of information.

This architecture ensures separation of concerns, making the system easier to maintain, debug, and scale. Each layer can be modified or enhanced independently without affecting other parts of the system.

3.3.2 User Interface Design

The user interface is designed with a focus on simplicity, clarity, and ease of use. The layout is structured to guide users through different functionalities without requiring technical knowledge.

The home page acts as the entry point, allowing users to select their role (Student, Warden, or Admin). Based on the selected role, users are directed to their respective dashboards.

The student dashboard provides a step-by-step workflow where users complete their profile, upload required documents, and proceed to hostel allocation and payment. The interface clearly displays available rooms based on eligibility and allocation rules, enabling informed decision-making. The billing section provides transparent fee details and payment options.

The admin/warden dashboard is designed to manage complex operations efficiently. It includes modules for:

- Student Records Management
- Billing Details and Payment Tracking
- Student Registrations
- Certificate Generation (Residence and No-Dues)
- Bulk Data Upload through Excel
- Hostel Block and Floor Allocation (Year-wise allocation control)

These features are organized using structured menus and sections, allowing easy navigation and quick access to functionalities. Notifications and alerts are used to inform users about successful actions, updates, or errors.

The interface is responsive and adapts to different devices such as desktops, laptops, and mobile phones, ensuring accessibility from anywhere. Consistent design elements such as layout, colours, and typography enhance the overall user experience.

3.3.3 Database Design

The database design ensures efficient storage, retrieval, and management of system data using a structured and relational approach. Data is organized into multiple interconnected entities to maintain consistency and avoid redundancy.

Key entities include Users, Students, Parent Details, Rooms, Allocations, Payments, Certificates, Billing Records, and Hostel Blocks/Floors. Each entity stores specific information relevant to system operations. For example, student profiles include personal, academic, and communication details, while allocation records link students to specific rooms and hostel blocks.

Relationships between entities are established to maintain data integrity. A student can have multiple payment and certificate records, while each student is associated with a single room allocation at a time. Parent details are linked to student records, and allocation data connects students with room and hostel information.

The design follows normalization principles to eliminate data duplication and ensure consistency. Efficient data retrieval is supported through optimized structuring and indexing of frequently accessed fields such as student identifiers and transaction records.

The database is designed to be scalable and flexible, allowing new entities or attributes to be added as the system evolves. This ensures that future enhancements, such as additional modules, increased data volume, or extended functionalities, can be incorporated without disrupting the existing system.

Design Considerations

The system design emphasizes efficiency, accuracy, and scalability. Data is structured to minimize redundancy and maintain consistency across modules. Validation mechanisms ensure correctness of inputs, while structured workflows improve operational efficiency. The system supports future expansion and enhancements without requiring major structural changes, making it a robust and adaptable solution for hostel management digitalization.

3.4 UML DESIGN OVERVIEW

UML Design – Digitalization of Hostel Management

Unified Modelling Language (UML) is used to represent the structure and behaviour of the Digitalization of Hostel Management in a clear and systematic way. It provides a standardized approach to visualize how different components of the system interact, how data flows across modules, and how various processes are executed. UML helps transform the conceptual design into a well-structured model, making it easier to understand, implement, and maintain the system efficiently.

The design begins by identifying the key users of the system, including students, wardens, and administrators, and analysing their interactions with system functionalities. These interactions are represented using a use case diagram, which illustrates how users engage with features such as registration, login, profile management, document upload, hostel block and floor allocation,

billing, payment processing, certificate generation, and record management. This helps clearly define system boundaries and ensures that all user requirements are properly captured.

The internal structure of the system is represented using a class diagram, which defines the major entities and their relationships. Core classes include user, student, parent details, hostel block, room, allocation, billing, payment, certificate, and records. Each class contains relevant attributes, and relationships between classes show how data is interconnected. This structural representation ensures logical organization of data and supports efficient data management.

The sequence of operations is illustrated using sequence diagrams, which show how different components interact over time to perform specific tasks. For example, when a student completes registration, selects a room, or makes a payment, the request flows through the interface to the system processing layer, where it is validated, processed, stored, and then a response is sent back to the user. This helps in understanding the order of execution and communication between different parts of the system.

The workflow of processes is represented using activity diagrams, which describe the step-by-step flow of actions involved in operations such as student registration, profile completion, hostel allocation, billing, and certificate generation. These diagrams include stages like input, validation, processing, and output, along with decision points based on conditions such as eligibility and availability. This provides a clear view of process flow and helps identify improvements.

Overall, the UML design provides a comprehensive understanding of the system by combining structural, behavioural, and interaction perspectives. It improves clarity in development, enhances communication among stakeholders, and supports the creation of a system that is organized, scalable, and easy to maintain.

3.4.1 Class Diagram

The Class Diagram consists of several key classes, each representing a core entity within the system. These classes include:

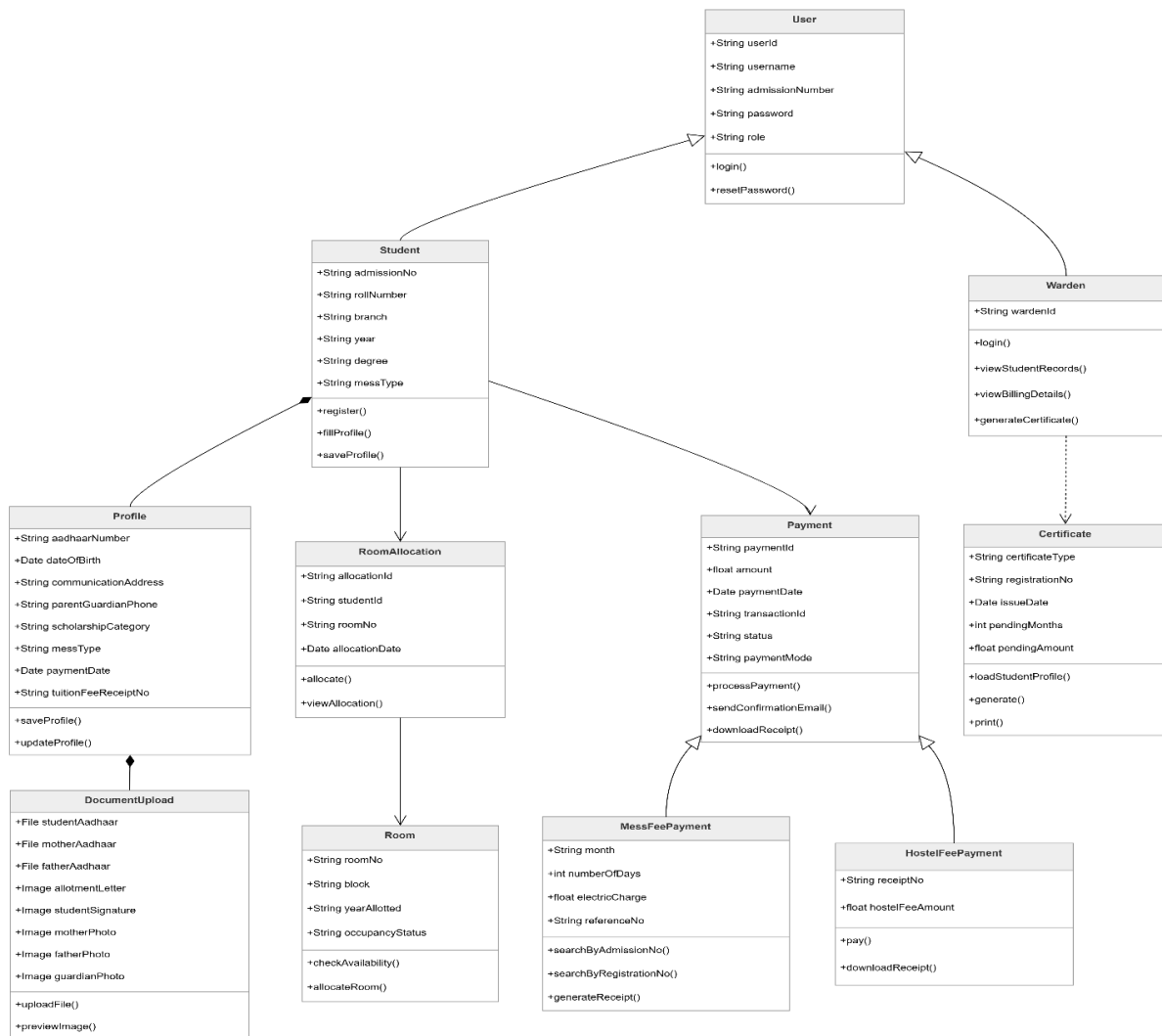


Figure 3.4.1 Class diagram of Digitalization of Hostel Management System

1. User Class

- Attributes: userId (INT), email (String), password (String), role (String).
- Methods: login(), logout(), register().
- Description: Represents the base user entity of the system. It provides authentication and access control functionalities and acts as the parent class for Student and Warden classes.

2. Student Class

- Attributes: studentId (INT), name (String), admissionNumber (String), branch (String), year (String).
- Methods: completeProfile(), selectRoom(), makePayment(), viewDetails().
- Description: A subclass of User, representing students in the system. It stores student-specific details and allows them to manage their profile, allocate rooms, and perform fee payments.

3. Warden Class

- Attributes: wardenId (INT), name (String), email (String).
- Methods: viewStudentRecords(), verifyPayments(), generateCertificates().
- Description: A subclass of User responsible for managing hostel operations. The warden monitors student data, room allocations, and payment details.

4. Profile Class

- Attributes: profileId (INT), studentId (INT), personalDetails (String), academicDetails (String), contactDetails (String).
- Methods: updateProfile(), viewProfile().
- Description: Stores complete student profile information including personal, academic, and communication details. It is linked directly to the Student class.

5. DocumentUpload Class

- Attributes: documentId (INT), studentId (INT), aadhaarFile (File), photo (File).
- Methods: uploadDocument(), viewDocument().
- Description: Handles uploading and storage of important documents such as Aadhaar and photographs. It is associated with the student profile.

6. Room Class

- Attributes: roomId (INT), roomNumber (String), block (String), capacity (INT), availabilityStatus (Boolean).
- Methods: checkAvailability(), updateRoomStatus().
- Description: Represents hostel rooms and their details. It helps in managing room availability and allocation.

7. RoomAllocation Class

- Attributes: allocationId (INT), studentId (INT), roomId (INT), allocationDate (Date), status (String).
- Methods: allocateRoom(), viewAllocation().
- Description: Manages the allocation of rooms to students based on availability and eligibility criteria.

8. Payment Class

- Attributes: paymentId (INT), studentId (INT), amount (FLOAT), paymentDate (Date), transactionId (String), status (String).
- Methods: processPayment(), viewPayment().
- Description: Represents payment transactions in the system and maintains financial records for each student.

9. MessFeePayment Class

- Attributes: month (String), numberOfDays (INT), messAmount (FLOAT).
- Methods: calculateMessFee(), processMessPayment().

- Description: A specialized class for handling mess fee payments, including calculation based on selected month and number of days.

10. HostelFeePayment Class

- Attributes: hostelFee (FLOAT), paymentStatus (String).
- Methods: processHostelFee(), verifyPayment().
- Description: Handles hostel fee transactions separately from mess fees, ensuring proper tracking of payments.

11. Certificate Class

- Attributes: certificateId (INT), studentId (INT), certificateType (String), issueDate (Date).
- Methods: generateCertificate(), downloadCertificate().
- Description: Manages the generation and storage of certificates such as residence certificate and no-dues certificate.

Relationships

- User to Student/Warden: A one-to-one relationship where a User can have a specific role as either Student or Warden.
- Student to Profile: A one-to-one relationship where each student has one profile containing detailed information.
- Student to Room Allocation: A one-to-one relationship where each student is allocated one room.
- Room to Room Allocation: A one-to-many relationship where one room can be allocated to multiple students (based on capacity).
- Student to Payment: A one-to-many relationship where a student can have multiple payment records.
- Payment to MessFeePayment/HostelFeePayment: An inheritance relationship where payment is categorized into different types.
- Student to Certificate: A one-to-many relationship where a student can generate multiple certificates.
- Student to Document Upload: A one-to-many relationship where a student can upload multiple documents.

3.4.2 Sequence Diagram

The sequence diagram illustrates the dynamic interactions within the Digitalization of Hostel Management System, focusing on key processes such as login, room allocation, and fee payment. It represents the communication between major components—Student, Interface, Processing Layer, Data Storage, Payment System, and Notification Service—highlighting the system’s runtime behaviour.

This diagram is designed based on the system's architecture and functional requirements, particularly authentication, room allocation, and online payment handling. It follows a client-server approach where user actions trigger a sequence of requests and responses, ensuring proper data flow, modularity, and efficient system performance.



Figure 3.4.2 Sequence diagram of Hostel Management System

The objective of this diagram is to clearly depict how user actions trigger a sequence of operations, including request handling, database interactions, payment processing, and notification services.

Sequence Flow

1. Login Flow

- o The process starts with the student entering credentials through the frontend.
- o The request is sent to the backend, which verifies the user by querying the database.
- o Upon successful authentication, a token is generated, and the user is redirected to the dashboard.

2. Room Allocation Flow

- o The student requests available rooms through the frontend interface.
- o The backend retrieves room availability from the database and returns the details.
- o After selection, the backend updates the allocation and confirms the booking.
- o A confirmation email is sent to the student via the email service.

3. Fee Payment Flow

- o The student initiates the payment process from the frontend.
- o The backend creates a payment order and communicates with the Razorpay gateway.
- o The payment is processed, and the status is returned to the backend.
- o The backend updates the payment details in the database and confirms the transaction.
- o A receipt is sent to the student via email, and the frontend displays the payment success message.

3.4.3 Use Case Diagram

A Use Case Diagram is a type of UML diagram that represents the functional requirements of a system by illustrating the interactions between users (actors) and the system's features (use cases). It provides a high-level view of what the system does from the user's perspective, showing how different actors engage with various functionalities within the system boundary. This diagram helps in identifying user roles, defining system behavior, and ensuring that all required functionalities are captured during the design phase.

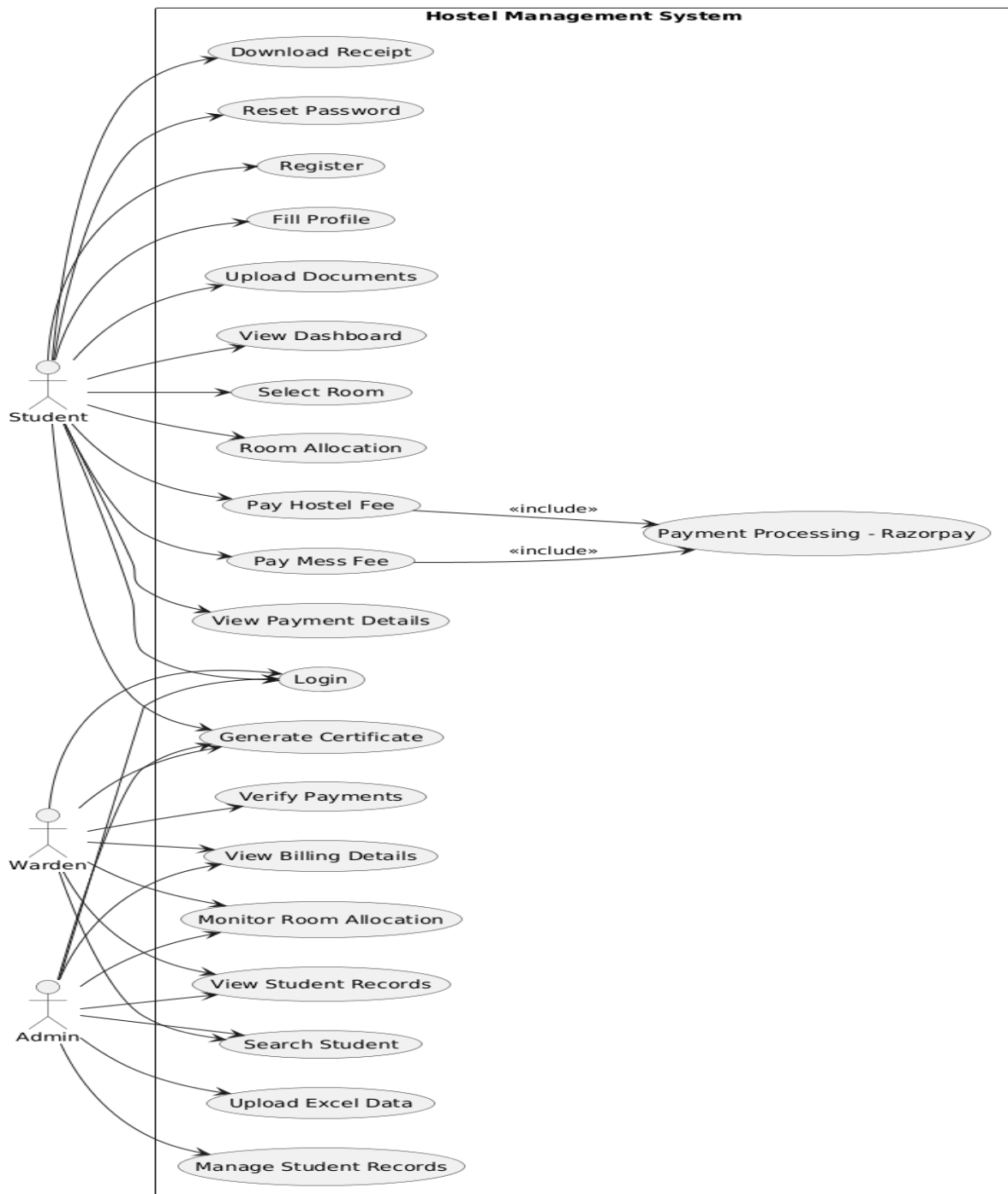


Figure 3.4.3 Use Case diagram of Digitalization of Hostel Management System

Actors

- Student: Primary user who performs registration, login, profile management, room selection, and fee payment.
- Warden: Manages student records, verifies payments, and oversees hostel operations.
- Admin: Controls system operations including student data management, room allocation, and bulk uploads.
- Payment System (Razor pay): External service used for secure fee transactions,

Use Cases

- Register: Students create an account with personal and academic details.
- Login: Users authenticate to access the system.
- Reset Password: Allows students to recover accounts via email.
- Manage Profile: Students enter and update profile and upload documents.
- Room Allocation: Students select rooms based on availability.
- Pay Hostel Fee / Pay Mess Fee: Students make online payments.
- Process Payment: Handles secure transactions through Razorpay.
- View Payment Details: Displays payment history and status.
- Download Receipt: Provides proof of successful payment.
- View Student Records: Admin/Warden can access student data.
- Verify Payments: Warden checks payment status.
- Generate Certificates: Admin/Warden generate required documents.
- Upload Excel Data: Admin uploads bulk student records.

Relationships

- Student interacts with core features like registration, profile, room allocation, and payments.
- Warden manages records, verifies payments, and generates certificates.
- Admin controls system data, uploads records, and monitors operations.
- Razor pay is linked with payment use cases for transaction processing.
- Payment use cases include payment processing as a mandatory step.

3.4.3 Activity Diagram

The activity diagram illustrates the workflow of the Digitalization of Hostel Management System, showing how key components Student, Interface, Processing Layer, Data Storage, Payment System, and Notification Service interact to complete major processes such as login, room allocation, and fee payment. It uses swim lanes to clearly represent the responsibilities of each component and the flow of control between them. The diagram represents the dynamic behaviour of the system, aligning with its architecture and functional requirements. It follows a structured flow where user actions trigger a sequence of operations that are processed, stored, and responded to accordingly. Decision points are included to handle conditions such as user validation and payment status, ensuring a smooth and controlled workflow.

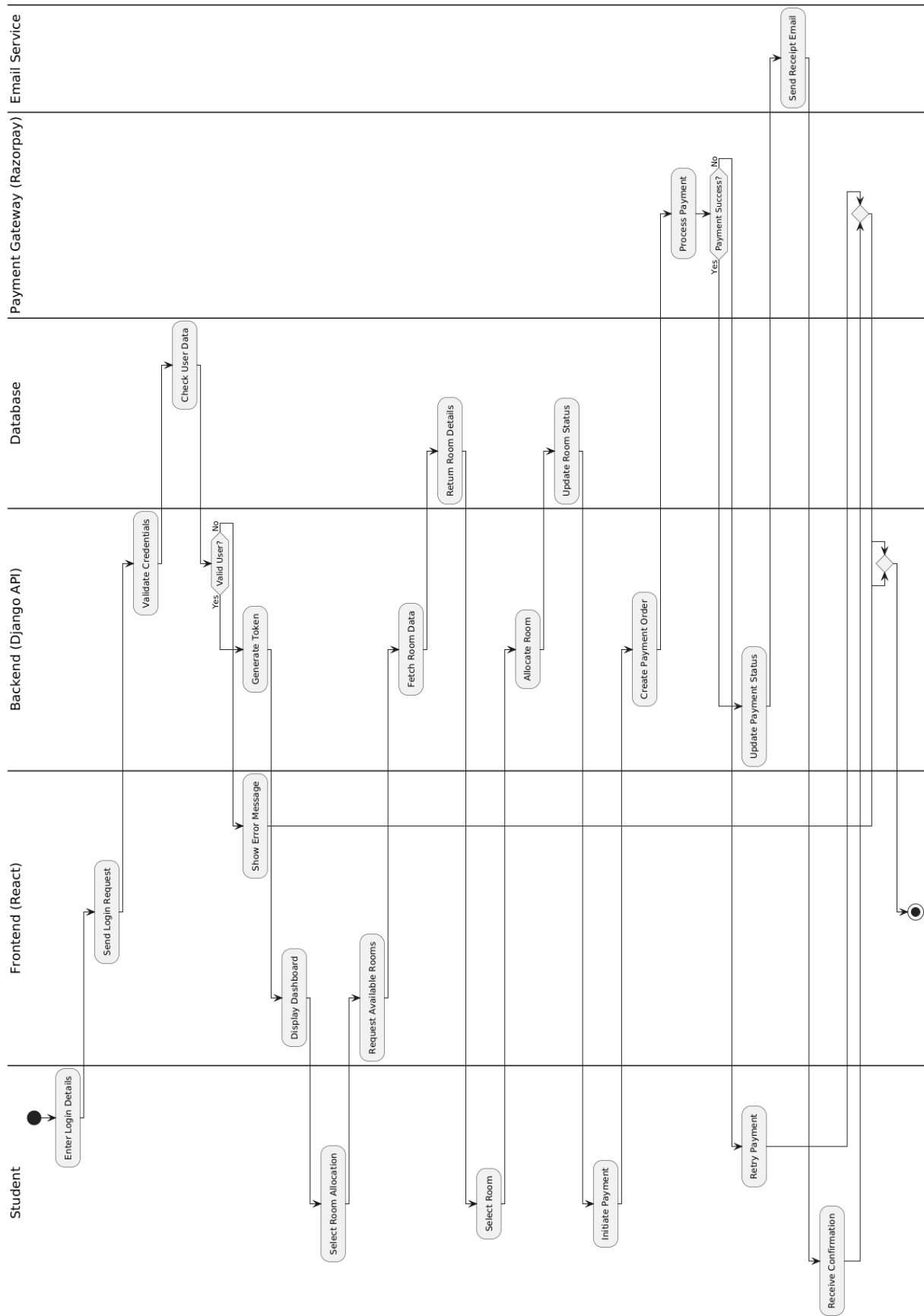


Figure 3.4.4 Activity diagram of Digitalization of Hostel Management System

Activity Flow

Login Process

- o The process begins with the student entering login details through the frontend.
- o The backend validates the credentials by checking user data in the database.
- o If the user is valid, a token is generated and the dashboard is displayed; otherwise, an error message is shown.

Room Allocation Process

- o The student selects the room allocation option and requests available rooms.
- o The backend fetches room data from the database and returns the available options.
- o The student selects a room, and the backend updates the allocation and room status in the database.

Fee Payment Process

- o The student initiates the payment process, and the backend creates a payment order.
- o The payment gateway processes the transaction and returns the payment status.
- o If the payment is successful, the backend updates the payment status and sends a receipt via email.
- o If the payment fails, the student is given the option to retry the payment.

The activity diagram provides a clear representation of the system workflow, showing how tasks are executed across different components and how decisions affect the flow of operations. It helps in understanding system behaviour, improving design clarity, and ensuring efficient implementation.

3.5 METHODOLOGY

Methodology

The development of the Digitalization of Hostel Management System follows a structured and systematic methodology to ensure effective planning, design, implementation, and validation. The approach is divided into multiple phases, allowing the system to be developed step-by-step while addressing both functional and non-functional requirements.

The process begins with requirement analysis, where the needs of users such as students, wardens, and administrators are identified. This includes analysing issues in the existing manual system like difficulty in maintaining records, lack of transparency in room allocation, and manual fee handling. Based on this, key features such as registration, login, profile management, room allocation, fee payment, and certificate generation are defined, along with requirements for security, performance, and usability.

In the system design phase, the overall architecture is planned using a layered structure consisting of presentation, processing, and data layers. The interface is designed to be simple and user-friendly, while the backend handles logic and data processing. UML diagrams such as use case, class, sequence, and activity diagrams are used to represent system workflows and structure. The database schema is also designed to ensure proper data organization and integrity.

The implementation phase involves developing the system modules based on the design. Functional components such as user management, room allocation, billing, and certificate generation are developed, along with integration of external services for payments and notifications. The system is built in a modular way to ensure flexibility and ease of development.

Next, testing and validation are performed to ensure all functionalities work correctly. This includes verifying user operations, data processing, and system responses under different conditions. Errors are identified and fixed to improve performance, reliability, and overall system stability.

Finally, the system is prepared for deployment and maintenance, ensuring it can be easily updated and scaled in the future. The modular design supports the addition of new features without affecting existing functionalities.

Overall, this methodology ensures that the Digitalization of Hostel Management System is developed in a structured, efficient, and reliable manner, resulting in a scalable, secure, and user-friendly solution for hostel management.

3.6 ALGORITHM

Step 1: Start

Step 2: Initialize System

Load the web application and initialize frontend (React), backend (Django API), and database (PostgreSQL).

Step 3: Establish Database Connection

Connect to the PostgreSQL database using backend configuration to enable data storage and retrieval.

Step 4: User Registration / Login

- If new user → Register by entering required details.
- If existing user → Enter login credentials.

Step 5: Validate User Credentials

Backend verifies user details from the database.

- If valid → Generate session/token and allow access.
- If invalid → Display error message and retry.

Step 6: Load Dashboard

After successful login, redirect the user to the respective dashboard based on role (Student/Warden/Admin).

Step 7: Profile Completion

Student enters personal, academic, communication, and parent details and uploads required documents.

Step 8: Save Profile Data

Store all entered details in the database and validate completeness.

Step 9: Select Room Allocation Option

Student requests available rooms from the system.

Step 10: Fetch Available Rooms

Backend retrieves room details from the database based on eligibility (year/block).

Step 11: Allocate Room

- Student selects a room.
- Backend updates allocation table and room availability.

Step 12: Choose Payment Type

Student selects either Hostel Fee or Mess Fee payment.

Step 13: Fetch Payment Details

Retrieve fee details such as amount, month, and number of days from the database.

Step 14: Initiate Payment

Create a payment request and redirect to Razorpay gateway.

Step 15: Process Payment

Payment gateway processes transaction and returns status.

Step 16: Update Payment Records

- If successful → Store transaction details in database.
- If failed → Display error and allow retry.

Step 17: Generate Receipt and Notification

Send confirmation email and provide downloadable receipt to the student.

Step 18: View Student Records

Admin/Warden searches student details using admission/registration number.

Step 19: View Billing Details

Retrieve and display payment history, transaction ID, and status.

Step 20: Verify Payments (Warden)

Check payment details and validate transactions.

Step 21: Generate Certificates

Generate residence, no-dues, or estimation certificates based on student data.

Step 22: Upload Excel Data (Admin)

Admin uploads bulk student data into the database.

Step 23: Logout

User logs out and session/token is cleared.

Step 24: Stop

IMPLEMENTATION

INTRODUCTION TO TECHNOLOGY

The implementation phase of the **Digitalization of Hostel Management System** converts the system design into a fully functional web-based application. This phase focuses on automating major hostel operations such as student registration, profile management, room allocation, fee payment, and certificate generation. By replacing traditional manual processes, the system provides a structured, efficient, and reliable digital solution for managing hostel activities.

The application is developed using a modern, layered architecture where the frontend, backend, and database interact seamlessly. The system also integrates external services to enhance functionality, ensuring smooth operation, secure transactions, and real-time communication. The implementation satisfies key requirements such as usability, performance, scalability, and security, making it suitable for real-time institutional use.

4.1 Tools and Technologies

The system is developed using a well-defined technology stack that ensures efficient communication between components and supports scalable and secure operations.

4.1.1 Backend Development

Python (Django & Django REST Framework):

Python 3.x is used for backend development due to its readability and flexibility. Django provides a robust framework for handling server-side operations such as user authentication, session management, request handling, and implementation of business logic. Django REST Framework (DRF) is used to build RESTful APIs that act as a bridge between the frontend and backend. It ensures secure data exchange, supports serialization of complex data, and allows efficient handling of HTTP requests and responses. Together, Django and DRF enable the system to process user inputs, enforce validation rules, and manage workflows like room allocation, billing, and certificate generation.

4.1.2 Frontend Development

React.js:

React is used to build an interactive and responsive user interface. It allows the development of reusable and modular components such as login forms, dashboards, profile pages, room selection interfaces, and payment modules. React's component-based architecture improves code maintainability and scalability. It also enhances user experience by providing fast rendering, smooth navigation, and real-time updates without reloading pages. This makes the system more dynamic and user-friendly for both students and administrators.

4.1.3 Database Management

PostgreSQL:

PostgreSQL is used as the relational database system to store and manage all application data

in a structured format. It handles data such as student profiles, hostel room details, allocations, payment records, certificates, and administrative logs. PostgreSQL ensures data integrity through constraints, relationships, and transaction management. It supports complex queries and large datasets efficiently. Integration with Django ORM (Object Relational Mapping) simplifies database interactions, allowing developers to perform operations without writing complex SQL queries.

4.1.4 External Integrations

Razor pay Payment Gateway:

Razor pay is integrated into the system to handle secure and seamless online payments for hostel and mess fees. It supports multiple payment methods including UPI, debit/credit cards, and net banking. The integration ensures safe transactions, real-time payment status updates, and automatic generation of payment confirmations and receipts.

Email Services:

Email services are used to send automated notifications to users. These include account verification, password reset links, payment confirmations, and other important alerts. This improves communication and ensures users stay informed about their activities within the system.

4.1.5 Development Environment

Operating System:

The system was developed and tested on Windows 10/11 using a local development setup.

IDE (Visual Studio Code):

Visual Studio Code is used as the primary development environment due to its lightweight nature and support for multiple extensions. It provides features like debugging, code formatting, and integration with Git, making development efficient.

API Testing Tool (Postman):

Postman is used to test REST APIs, verify request-response cycles, and ensure proper communication between frontend and backend systems.

Version Control (Git):

Git is used for version control and collaboration, allowing developers to track changes, manage code versions, and work efficiently in a team environment.

These technologies were chosen for their robustness, scalability, and compatibility with modern web development standards. The integration of React, Django, and PostgreSQL, along with external services, ensures that the Digitalization of Hostel Management System is secure, efficient, and capable of handling real-time hostel operations with high reliability.

4.2 SAMPLE CODE

HOME PAGE:

```
import React, { useState, useEffect } from "react";
import { useNavigate } from "react-router-dom";

function Homepage() {
  const navigate = useNavigate();
  const [loading, setLoading] = useState(true);
  const [booking, setBooking] = useState(null);

  useEffect(() => {
    fetchBooking();
  }, []);

  const fetchBooking = async () => {
    try {
      const token = localStorage.getItem('access');
      if (!token) {
        setLoading(false);
        return;
      }

      const res = await fetch('http://127.0.0.1:8000/api/app/my-booking/', {
        headers: {
          'Authorization': `Bearer ${token}`,
          'Content-Type': 'application/json',
        },
      });

      if (!res.ok) {
        console.log("API error:", res.status);
        setLoading(false);
        return;
      }

      const data = await res.json();
      console.log("Booking data:", data);
      if (data?.booking) {
        setBooking(data.booking);
      }
    } catch (err) {

```

```

    console.log("Fetch error:", err);
  } finally {
    setLoading(false);
  }
};

const handleLogout = () => {
  if (window.confirm("Are you sure you want to logout?")) {
    localStorage.removeItem("access");
    localStorage.removeItem("refresh");
    localStorage.removeItem("user");
    localStorage.removeItem("role");
    localStorage.clear();
    window.dispatchEvent(new Event("logout"));
    navigate("/");
  }
};

if (loading) {
  return (
    <div className="min-h-screen bg-slate-50 flex items-center justify-center">
      <p className="text-slate-500">Loading...</p>
    </div>
  );
}

return (
  <div className="min-h-screen bg-slate-50 flex flex-col">

    {/* --- MAIN CONTENT AREA --- */}
    <div className="flex-1 flex flex-col items-center justify-center p-4">
      <h1 className="text-xl font-bold text-slate-400 mb-10 tracking-[0.2em] uppercase">
        Select Service
      </h1>

      <div className="flex flex-col md:flex-row gap-6 w-full max-w-4xl">
        {/* Room Allocation Button */}
        <button
          className="flex-1 group bg-white border-2 border-slate-100 p-10 rounded-4xl
transition-all duration-300 hover:border-[#002147] hover:shadow-2xl hover:-translate-y-2"
          onClick={() => navigate("/room-booking")}>

```

PROFILE PAGE:

```
import React, { useState, useEffect } from "react";
import { useNavigate } from "react-router-dom";
import PropTypes from "prop-types";
import { studentApi as api } from "../services/api";
import { toast } from 'react-toastify';

function Profile() {
  const navigate = useNavigate();

  const [formData, setFormData] = useState({
    full_name: "", dob: "", aadhar_no: "", aadhar_pdf: null, student_photo: null,
    admission_no: "", reg_no: "", roll_no: "", degree: "", branch: "", year: "",
    admission_date: "", caste: "", mobile: "", email: "", address: "",
    father_name: "", father_phone: "", father_aadhar_no: "", father_aadhar_pdf: null,
    father_photo: null,
    mother_name: "", mother_phone: "", mother_aadhar_no: "", mother_aadhar_pdf: null,
    mother_photo: null,
    amount: "13000",
  });

  const [preview, setPreview] = useState({
    student: "", father: "", mother: "", aadhar: "", father_aadhar: "", mother_aadhar: ""
  });

  // Auto-fetch profile on mount using JWT token
  useEffect(() => {
    fetchProfile();
  }, []);

  const fetchProfile = async () => {
    try {
      const token = localStorage.getItem('access');
      const userStr = localStorage.getItem("user");

      console.log("Token exists:", !!token);
      console.log("User data:", userStr);

      if (!token) {
        navigate("/login/student");
        return;
      }
    }
  }
}
```

```

}

let studentId = "";
if (userStr) {
  const user = JSON.parse(userStr);
  studentId = user.admission_no || user.admission_number || user.username;
  console.log("Student ID:", studentId);
}

if (!studentId) {
  toast.error("User not found. Please login again.");
  navigate("/login/student");
  return;
}

// Fetch using JWT auth
const response = await fetch(
  `http://127.0.0.1:8000/api/student/get-student-profile/?admission_no=${studentId}`,
  {
    headers: {
      'Authorization': `Bearer ${token}`,
      'Content-Type': 'application/json',
    },
  }
);
setFormData((prev) => ({
  ...prev,
  full_name: student.full_name || prev.full_name,
  dob: student.dob || prev.dob,
  aadhar_no: student.aadhar || student.aadhar_no || prev.aadhar_no,
  admission_no: student.admission_no || prev.admission_no,
  reg_no: student.reg_no || prev.reg_no,
  roll_no: student.roll_no || prev.roll_no,
  degree: student.degree || prev.degree,
  branch: student.branch || prev.branch,
  year: student.class_yr || student.year || prev.year,
  admission_date: student.admission_date || prev.admission_date,
  caste: student.caste || prev.caste,
  mobile: student.mobile || prev.mobile,
  email: student.email || prev.email,
  address: student.address || prev.address,
  father_name: student.father_name || prev.father_name,
  father_phone: student.father_phone || prev.father_phone,

```

```

        father_aadhar_no: student.father_aadhar_no || student.father_aadhar ||
prev.father_aadhar_no,
        mother_name: student.mother_name || prev.mother_name,
        mother_phone: student.mother_phone || prev.mother_phone,
        mother_aadhar_no: student.mother_aadhar_no || student.mother_aadhar ||
prev.mother_aadhar_no,
        amount: student.amount || prev.amount,
    }));

    setPreview({
        student: getImageUrl(student.student_photo),
        father: getImageUrl(student.father_photo),
        mother: getImageUrl(student.mother_photo),
        aadhar: getImageUrl(student.aadhar_pdf || student.aadhar_file),
        father_aadhar: getImageUrl(student.father_aadhar_pdf || student.father_aadhar_file),
        mother_aadhar: getImageUrl(student.mother_aadhar_pdf ||
student.mother_aadhar_file),
    });

    console.log("Preview URLs set:", {
        student: getImageUrl(student.student_photo),
        father: getImageUrl(student.father_photo),
        mother: getImageUrl(student.mother_photo),
    });

```

ROOM SELECTION

```

import React, { useState, useEffect } from "react";
import { useNavigate } from "react-router-dom";
import { toast } from 'react-toastify';
import { hostelApi as api } from "../services/api";

const RoomBooking = () => {
    const navigate = useNavigate();
    const [loading, setLoading] = useState(true);
    const [error, setError] = useState("");
    const [showConfirmModal, setShowConfirmModal] = useState(false);
    const [confirmAction, setConfirmAction] = useState(null);
    const [activeFloor, setActiveFloor] = useState(0);

    const [activeBooking, setActiveBooking] = useState(null);

```

```

const [blockInfo, setBlockInfo] = useState(null);
const [floors, setFloors] = useState([]);
const [allowedFloors, setAllowedFloors] = useState([]);

// Force room disable when user has confirmed booking
const hasConfirmedBooking = activeBooking?._status === 'confirmed';

useEffect(() => {
  checkInitialStatus();
}, []);

const checkInitialStatus = async () => {
  try {
    console.log("Checking booking status...");
    const bookingRes = await api.get("/my-booking/");
    console.log("Booking response:", bookingRes.status, bookingRes.data);

    // Handle 404 as "no booking" - not an error
    if (bookingRes.status === 404) {
      console.log("No booking found - 404 is OK");
      // Continue to load room list
    } else if (bookingRes.data?.status === 'confirmed' &&
bookingRes.data?.booking) {
      console.log("Confirmed booking found:", bookingRes.data.booking);
      setActiveBooking({
        ...bookingRes.data.booking,
        _status: 'confirmed'
      });
      setLoading(false);
      setError("");
      return; // STOP - don't load room list
    } else if (bookingRes.data?.status === 'pending') {
      console.log("Pending booking found");
      setActiveBooking({
        id: bookingRes.data.booking_id,
        block_name: blockInfo?.display_name || "Reserved Block",
        room_number: "Room Locked for Payment",
        _status: 'pending'
      });
    }
  } catch (error) {
    console.error("Error checking booking status:", error);
  }
};

```

TESTING

TESTING

5.1 INTRODUCTION

Testing is a crucial phase in the development of the Digitalization of Hostel Management System, ensuring that the application functions accurately, efficiently, and securely across all modules. It validates the complete workflow from input to output, confirming that features such as registration, login, room allocation, fee processing, and administrative operations perform as expected under different conditions.

The testing process focuses on proper data handling, reliable system behaviour, and secure access control while identifying and resolving potential errors. Both functional and non-functional aspects, including usability, performance, and security, are evaluated to ensure system stability. Testing was conducted in a local environment, confirming that the system is stable, user-friendly, and ready for real-time hostel management operations.

Testing objectives:

The primary goal of testing is to systematically identify any errors in the system while minimizing the time and effort required. In formal terms, testing is the process of executing a program to detect any faults or errors within it. The objectives of testing are:

- **Error Detection:** The primary goal of testing is to identify as many errors as possible within the system.
- **Effective Test Cases:** A good test case is one that is designed to uncover errors if they exist, ensuring comprehensive coverage of all functionalities.
- **Quality Assurance:** Testing ensures that the software meets the expected quality standards, both in terms of functionality and user experience.
- **Efficiency in Error Discovery:** The testing process aims to be efficient, ensuring that errors are detected with minimal resources and time.

Types of Tests

Unit testing:

A unit is the smallest piece of source code that can be tested. It is also known as a module which consists of numerous lines of code that are processed by a single programmer. The key purpose of performing unit testing is to expose that a particular unit doesn't satisfy the specified functional requirements and also to show that the

structural implementation is not like to the projected structure designed.

Integration testing:

Tests are intended to test incorporated programming segments to figure out whether they really keep running as one system. Testing is occasion driven and is more worried with the fundamental result of screens or fields. Reconciliation tests exhibit that in spite of the fact that the parts were separately fulfillment, as appeared by effectively unit testing, the blend of segments is right and comprised. Integration testing is specifically aimed at revealing the problems that rise from the mixture of components.

Functional testing:

Functional tests give efficient challenges that capacities tried are accessible as determined by the business and specialized necessities, framework documentation, and client manuals. Association and arrangement of practical tests is centered around prerequisites, key capacities, or unique experiments. Likewise, efficient scope relating to recognize Business procedure streams; information fields, predefined procedures, and progressive procedures must be considered for testing. Before utilitarian testing is finished, extra tests are distinguished and the powerful estimation of current tests is resolved.

System Testing:

System testing guarantees that the entire coordinated programming framework meets prerequisites. It tests a design to guarantee known and unsurprising results. A sample of framework testing is the arrangement situated framework combination test. Framework testing depends on procedure portrayals and streams, stressing pre-driven procedure connections and mix focuses.

White Box Testing:

It is a testing in which the product analyzer has information of the internal workings, structure and dialect of the product, or if nothing else its motivation. It is reason. It is utilized to test ranges that can't be gotten a handle on from a discovery level.

Black Box Testing:

It is the testing the product with no information of within workings, structure or dialect of the part being tried. Discovery tests, as most different sorts of tests, must be composed from a complete source report, for example, prerequisite or necessities archive, for example, determination or necessities record. It is a trying in which the product under test is dealt with, as a discovery. you can't "see" into it. The test gives inputs and reacts to yields without considering how the product functions.

5.2 TEST CASES:

Test Case 1: User Login

Description: Verify login with valid credentials.

Preconditions: User registered

Test Steps:

1. Open login page
2. Enter valid credentials
3. Click "Login"

Expected Output: Redirect to dashboard

Actual Result: Login successful

Status: Pass

Test Case 2: Invalid Login

Description: Verify login fails with wrong password.

Preconditions: User exists

Test Steps:

1. Enter incorrect password
2. Click "Login"

Expected Output: Error message displayed

Actual Result: Login denied

Status: Pass

Test Case 3: Student Registration

Description: Verify new user registration.

Preconditions: User not registered

Test Steps:

1. Enter valid details
2. Click "Register"

Expected Output: Account created

Actual Result: Registration successful

Status: Pass

Test Case 4: Duplicate Registration

Description: Verify duplicate entries are restricted.

Preconditions: User already exists

Test Steps:

1. Enter existing admission number
2. Click "Register"

Expected Output: Error message

Actual Result: Duplicate blocked

Status: Pass

Test Case 5: Profile Submission

Description: Verify complete profile submission.

Preconditions: User logged in

Test Steps:

1. Enter all required details
2. Click "Save"

Expected Output: Profile saved

Actual Result: Data stored successfully

Status: Pass

Test Case 6: Room Allocation

Description: Verify room selection process.

Preconditions: Rooms available

Test Steps:

1. Select room

2. Confirm

Expected Output: Room allocated

Actual Result: Room assigned

Status: Pass

Test Case 7: Room Availability Check

Description: Verify full rooms cannot be selected.

Preconditions: Room capacity full

Test Steps:

1. Select full room

Expected Output: Error message

Actual Result: Selection blocked

Status: Pass

Test Case 8: Fee Payment

Description: Verify successful payment process.

Preconditions: Fees generated

Test Steps:

1. Proceed to payment

2. Complete transaction via Razor pay

Expected Output: Payment success + receipt

Actual Result: Payment completed

Status: Pass

Test Case 9: Payment Failure Handling

Description: Verify system handles failed transactions.

Preconditions: Payment initiated

Test Steps:

1. Cancel payment

Expected Output: Failure message

Actual Result: Payment not recorded

Status: Pass

Test Case 10: Role-Based Access Control

Description: Verify unauthorized access is restricted.

Preconditions: User logged in

Test Steps:

1. Try accessing restricted module

Expected Output: Access denied

Actual Result: Access blocked

Status: Pass

RESULT AND ANALYSIS

RESULT AND ANALYSIS

The implementation of the Digitalization of Hostel Management System has resulted in a fully functional web-based application designed to streamline hostel operations, including room allocation, fee management, and digital record maintenance. The system reduces reliance on manual record-keeping by providing a centralized and organized platform.

It offers a clean, role-based interface tailored for administrators and wardens, enabling them to efficiently manage student details, monitor records, and generate necessary certificates, ensuring a smooth and reliable hostel management process.

RESULTS

HOME PAGE :

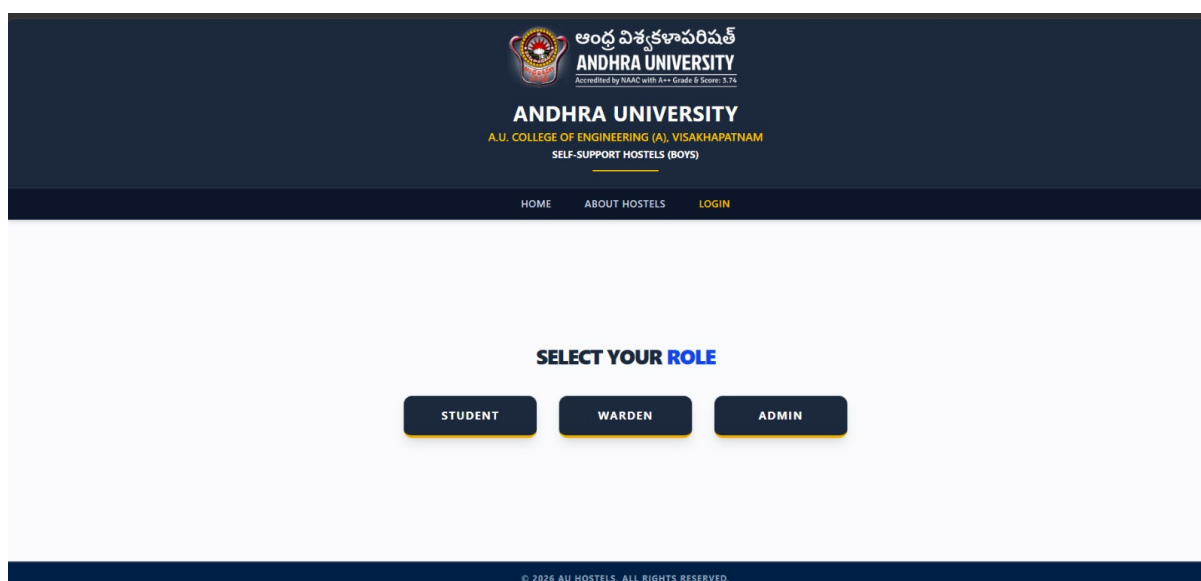


Figure No: 6.1 Home Page

As shown in the above picture, the home page of the Digitalization of Hostel Management System serves as the main entry point with a clean and structured interface. It includes a header with institutional branding and a navigation bar for easy access to key sections like Home, About Hostels, and Login.

The central section highlights role-based access with options such as Student, Warden, and Admin. The design is simple and user-friendly, enabling quick access to relevant functionalities.

6.1 STUDENT:

REGISTRATION PAGE:

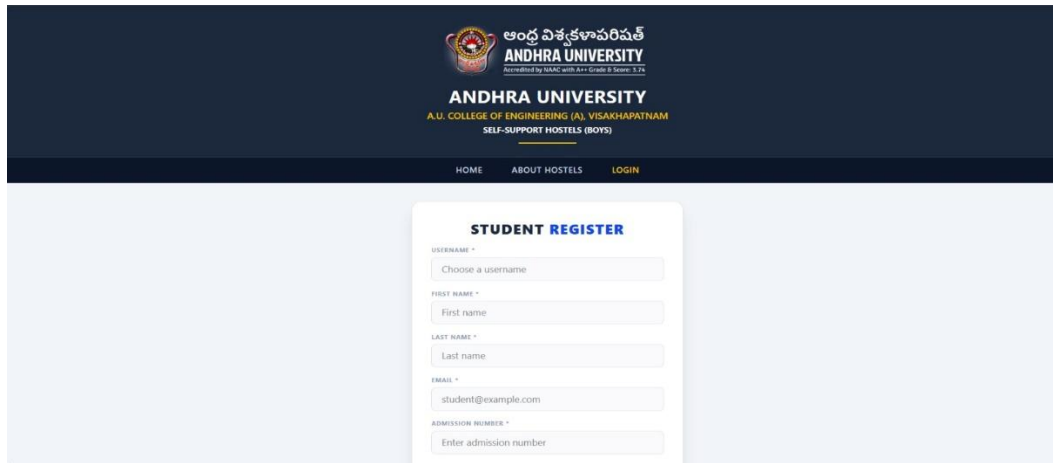
The screenshot shows the 'STUDENT REGISTER' form on the Andhra University website. The header includes the university's name in Telugu and English, its accreditation (NAAC A++ Grade B, Score: 3.74), and the specific institution: 'A.U. COLLEGE OF ENGINEERING (A), VISAKHAPATNAM SELF-SUPPORT HOSTELS (BOYS)'. Navigation links for 'HOME', 'ABOUT HOSTELS', and 'LOGIN' are present. The registration form itself has the following fields: 'USERNAME' (with a placeholder 'Choose a username'), 'FIRST NAME' (placeholder 'First name'), 'LAST NAME' (placeholder 'Last name'), 'EMAIL' (placeholder 'student@example.com'), and 'ADMISSION NUMBER' (placeholder 'Enter admission number').

Figure No 6.1.1 Registration Page

In the above picture , the student registration page enables new users to create an account by providing essential personal and academic details, including username, first name, last name, email, admission number (for first-year students), phone number, year of study (selected from a dropdown), and branch. It also requires users to set and confirm a password to ensure secure authentication. The form is clearly structured with proper input fields and validation, making it easy to use while ensuring accurate data collection for accessing hostel-related services.

LOGIN PAGE:

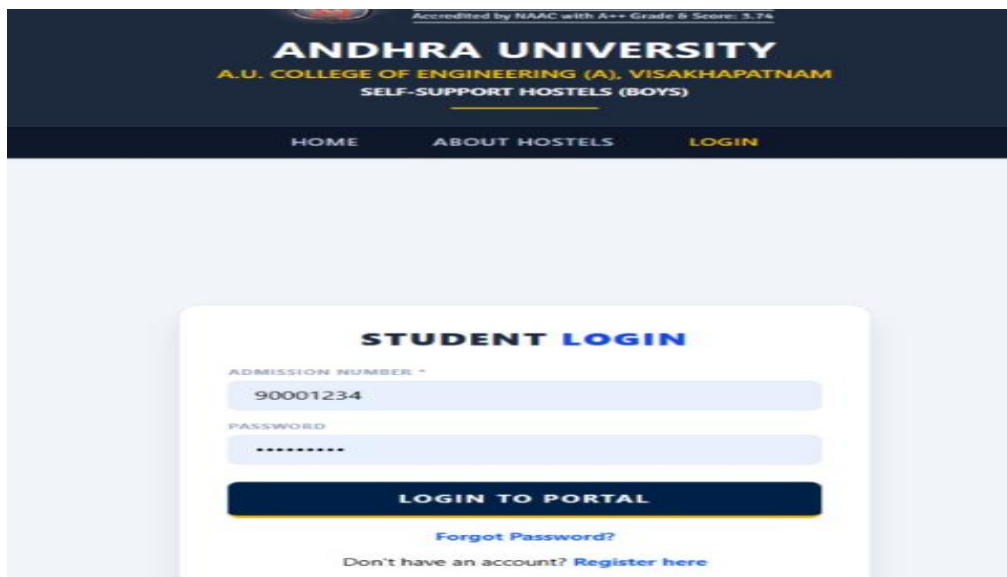
The screenshot shows the 'STUDENT LOGIN' form. The header is identical to the registration page. The login form contains two input fields: 'ADMISSION NUMBER' (with the example value '90001234') and 'PASSWORD' (masked with asterisks). Below these fields is a dark blue button labeled 'LOGIN TO PORTAL'. At the bottom of the form, there are two links: 'Forgot Password?' and 'Don't have an account? Register here'.

Figure No 6.1.2 Login Page

The **Student Login Page** allows registered students to securely access the system. Users

enter their Admission Number and Password and click “Login to Portal” to proceed. It also includes options for Forgot Password and new user registration. The interface is simple and user-friendly for quick and secure access.

STUDENT PROFILE :

ANDHRA UNIVERSITY
College of Engineering (A) Hostel

Admission Fee - 13000/-

PERSONAL INFORMATION

FULL NAME
NAMALA EEKSHITHA

DATE OF BIRTH
05-09-2005

AADHAAR NUMBER
891133344571

AADHAAR (PDF)
Aadhar Card.pdf CHANGE

CASTE
BC-B

ACADEMIC DETAILS

ADMISSION NO
322107311030

REG NO
322107311030

ROLL NO
322107311030

ADMISSION DATE
dd-mm-yyyy

DEGREE
B.Tech

BRANCH
INF

YEAR
4/4

COMMUNICATION DETAILS

MOBILE NUMBER
08522077666

EMAIL ADDRESS
eekshithanamala3@gmail.com

FULL ADDRESS
57-8-5,GOKUL NAGAR

B.Tech
INF
4/4

COMMUNICATION DETAILS

MOBILE NUMBER
08522077666

EMAIL ADDRESS
eekshithanamala3@gmail.com

FULL ADDRESS
57-8-5,GOKUL NAGAR

FATHER

FATHER NAME
N V S N VARA PRASADA RAO

FATHER PHONE
9840071896

FATHER AADHAAR NO
567890134142

AADHAAR (PDF)
eekshitha-namal... CHANGE

MOTHER

MOTHER NAME
N MAHALAKSHMI

MOTHER PHONE
7890132412

MOTHER AADHAAR NO
890765431234

AADHAAR (PDF)
MCQ-Question-an... CHANGE

SAVE PROFILE

Figure No 6.1.3 Student Profile Page

The **Profile Page** allows students to view and manage their personal, academic, and contact details in a structured format. Users can update information, upload documents, and add guardian details as required. It includes a save/update option to securely store changes. Overall, it acts as a centralized record supporting verification and room allocation processes.

50

STUDENT DASHBOARD:

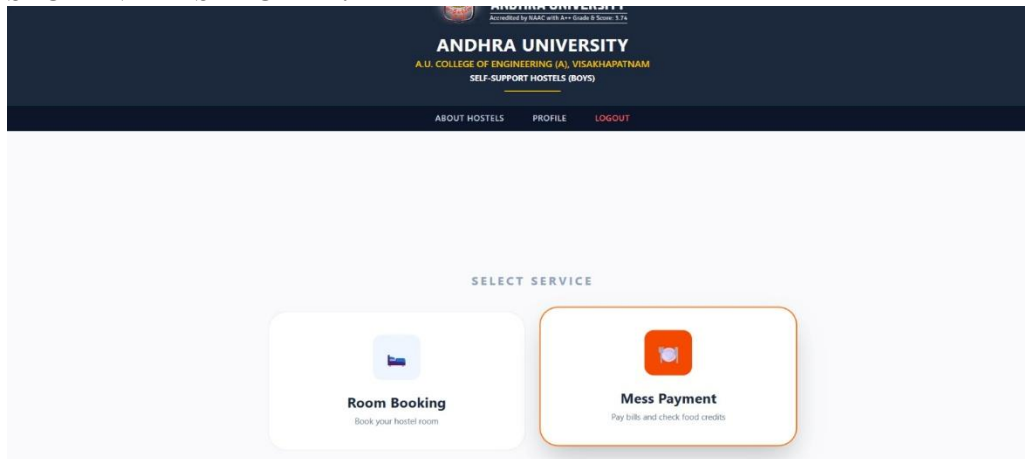


Figure 6.1.4 Student Dashboard

The student dashboard acts as the main control panel after login, providing quick access to essential hostel services. It presents a simple and intuitive interface with clearly defined options such as room booking and mess payment. The room booking section allows students to view and select available rooms, while the mess payment section enables them to pay bills and track their food-related expenses. The layout is clean and user-friendly, ensuring that students can easily navigate between services. Overall, the dashboard centralizes key functionalities, making hostel management tasks convenient and efficient for students.

ROOM BOOKING / ALLOCATION:

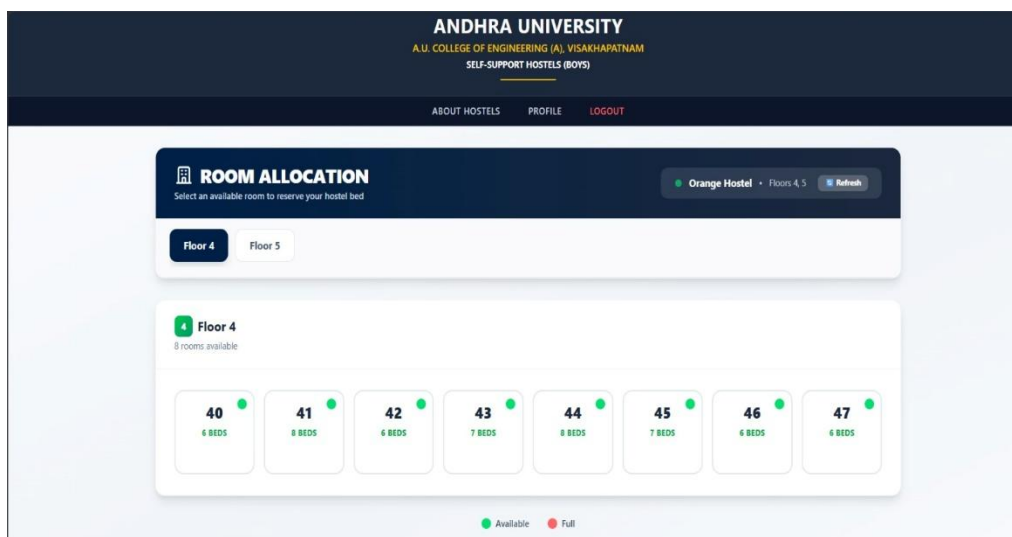


Figure No 6.1.5 Room Allocation Page

The room allocation page allows students to select and reserve a hostel room based on availability. Rooms are displayed according to the student's assigned block and year of study, ensuring that only eligible options are shown. The interface is organized floor-wise, enabling users to easily navigate between different floors and view available rooms along with details like room number and bed capacity. Available rooms are clearly indicated, and students can choose their preferred room and proceed with booking. This structured approach makes the room selection process simple, transparent, and efficient.

ROOM BOOKING CONFIRMATION:

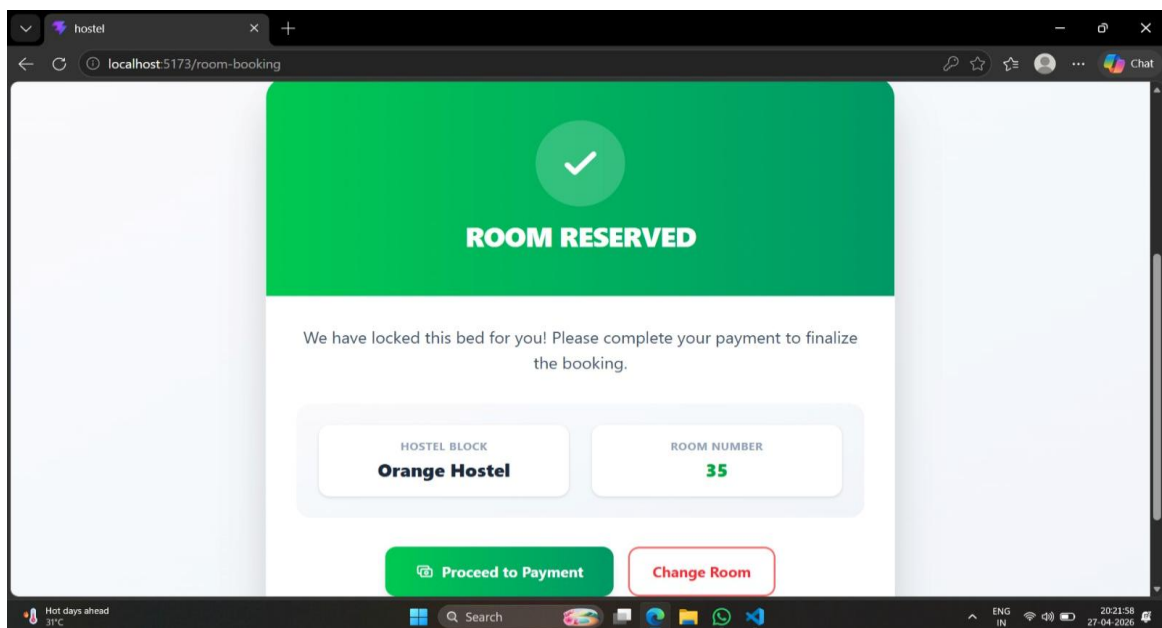


Figure No 6.1.5 Room Booking Confirmation

This page displays the confirmation of a successful room selection. Once a student chooses a room, the system temporarily reserves a bed and shows the selected hostel block and room number. It clearly informs the user that the booking is not yet complete and requires payment to finalize the allocation. The page also provides two options: to proceed with payment and confirm the booking, or to change the selected room if needed. This step ensures clarity and gives students a chance to review or modify their choice before final confirmation.

FEE PAYMENT :

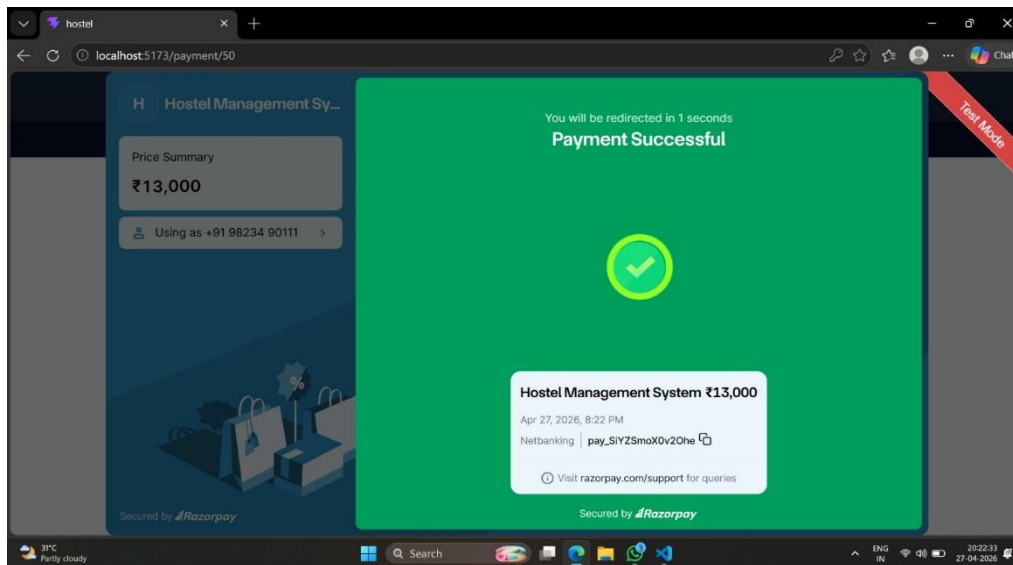


Figure No 6.8 Fee Payment

This page shows the confirmation of a successful payment for hostel room booking. After completing the transaction, a success message is displayed along with the payment details, including the amount paid and transaction reference. It indicates that the booking process has been completed and the room is now officially allocated to the student. The page also mentions that the user will be redirected automatically, ensuring a smooth transition back to the system. Overall, it provides clear confirmation and assurance that the payment and booking have been successfully processed.

6.2 ADMIN

LOGIN PAGE:

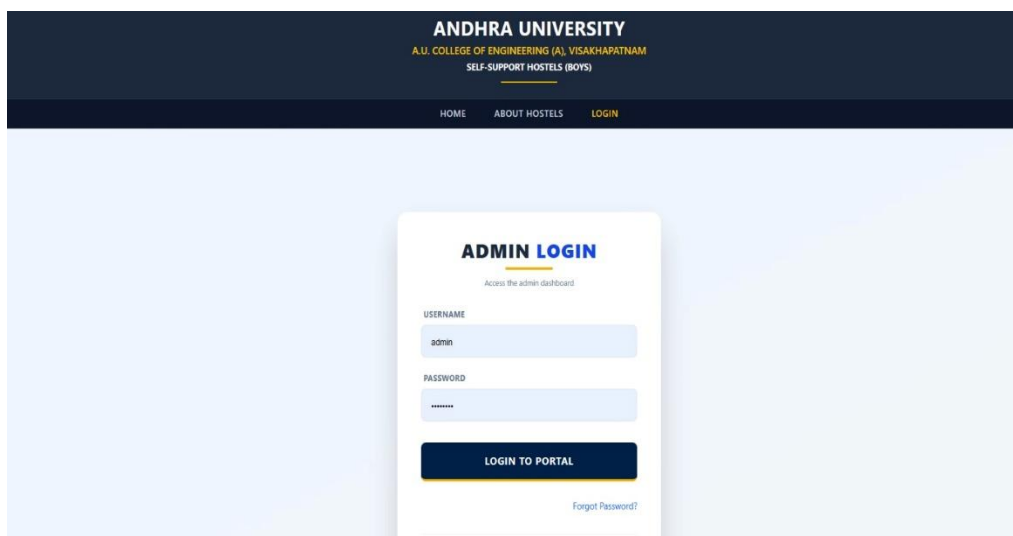


Figure No 6.2.1 Admin Login

The admin login page provides secure access for authorized personnel to manage hostel operations. Both wardens and administrators log in using their assigned ID and password. This ensures role-based access control, where each user is directed to their respective dashboard with specific permissions. Wardens can manage student records, room allocations, and monitor hostel activities, while admins have broader control over system management, user handling, and overall hostel administration. The login system ensures security, restricted access, and efficient management of hostel services.

ADMIN DASHBOARD:

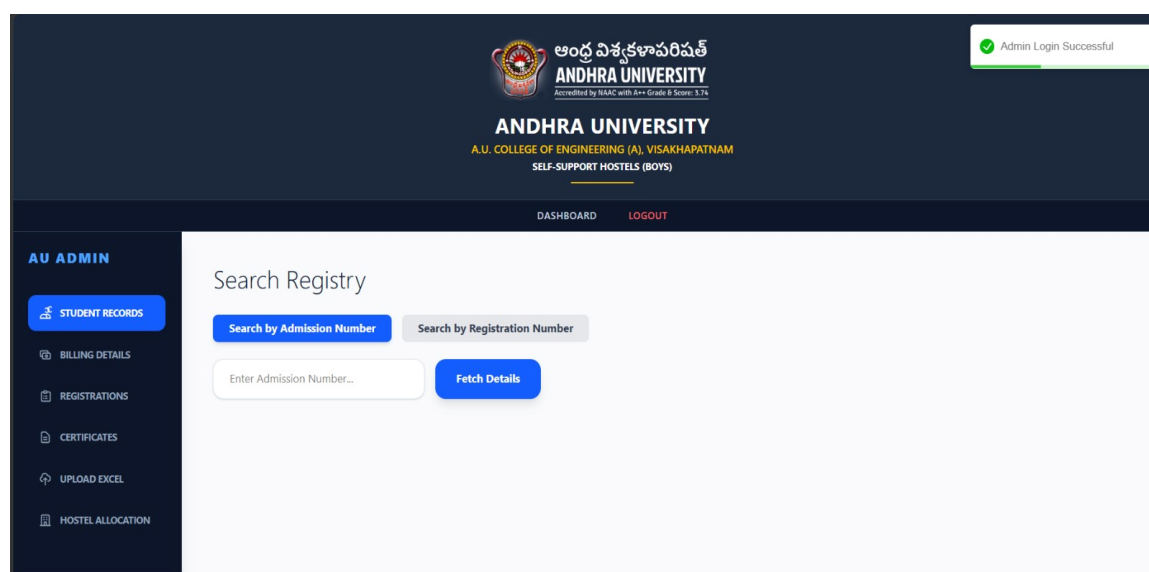


Figure No 6.2.2 Admin Dashboard

The admin dashboard serves as the central control panel for managing all hostel operations efficiently. It provides a sidebar menu with key modules such as student records, billing details, registrations, certificates, data upload, and hostel allocation. The main section includes a search registry feature, allowing the admin to quickly retrieve student information using either admission number or registration number. This enables easy access to student details for verification, updates, and management. The interface is designed to be organized and user-friendly, helping the admin handle large amounts of data, monitor activities, and perform administrative tasks smoothly and effectively.

STUDENTS RECORDS:

The screenshot shows the 'AU ADMIN' dashboard with a sidebar menu containing 'STUDENT RECORDS', 'BILLING DETAILS', 'REGISTRATIONS', 'CERTIFICATES', 'UPLOAD EXCEL', and 'HOSTEL ALLOCATION'. The 'STUDENT RECORDS' section is active, displaying a 'Search Registry' form with two search options: 'Search by Admission Number' and 'Search by Registration Number'. The 'Search by Admission Number' option is selected, and the input field contains '323506402161'. A 'Fetch Details' button is next to the input field. Below the search form, the student details are displayed in four columns: 'ACADEMIC PROFILE', 'PERSONAL INFORMATION', 'ROOM DETAILS', and 'FAMILY & MEDIA'. The 'ACADEMIC PROFILE' column shows the student's name, admission and registration numbers, roll number, year/branch, and category. The 'PERSONAL INFORMATION' column shows the student's mobile number, email ID, Aadhaar number, date of birth, caste, and fee amount. The 'ROOM DETAILS' column shows the room number, block name, floor, bed number, room type, and boarding type. The 'FAMILY & MEDIA' column shows the father's name, mother's name, guardian, and a section for student and father photos.

ACADEMIC PROFILE	PERSONAL INFORMATION	ROOM DETAILS	FAMILY & MEDIA
FULL NAME GUDIBANDI SURYA PRAKASH REDDY	MOBILE 9505858328	ROOM NUMBER 2	FATHER NAME Hari krishna Reddy (9948096391)
ADMISSION NO 323506402161	EMAIL ID Not Provided	BLOCK NAME meta	MOTHER NAME Not Provided
REGISTRATION NO 323506402161	AADHAAR NO Not Provided	FLOOR N/A	GUARDIAN Not Provided
ROLL NUMBER 323506402161	DATE OF BIRTH 2005-08-07	BED NUMBER N/A	STUDENT FATHER
YEAR/AL BRANCH 2 - CSE	CASTE REDDY	ROOM TYPE N/A	
CATEGORY Not Specified	FEES AMOUNT ₹13250	BOARDING TYPE N/A	

Figure No 6.2.3 Student Records search registry

This page in the admin dashboard displays detailed student information after performing a search using an admission or registration number. Once the admin fetches the data, the system organizes the student's details into clearly separated sections such as academic profile, personal information, room details, and family information. Each section presents relevant data in a structured format, allowing the admin to quickly view and verify records like academic details, contact information, hostel allocation status, and guardian details. This layout helps in efficient monitoring, validation, and management of individual student records within the hostel system.

BILLING DETAILS:

The screenshot shows the 'AU ADMIN' dashboard with a sidebar menu containing 'STUDENT RECORDS', 'BILLING DETAILS', 'REGISTRATIONS', 'CERTIFICATES', 'UPLOAD EXCEL', and 'HOSTEL ALLOCATION'. The 'BILLING DETAILS' section is active, displaying a 'Billing Details' form with two search options: 'Search by Admission Number' and 'Search by Registration Number'. The 'Search by Admission Number' option is selected, and the input field contains '323506402161'. A 'Fetch Billing' button is next to the input field. Below the search form, the 'Student Summary' section displays the student's name, admission and registration numbers, room number, category, total paid, and total months. Below the summary, a table displays the billing details for the student, including the month, days, electric charge, mess charge, service charge, net demand, collection, date, and status.

Month	Days	Electric Charge	Mess Charge	Service Charge	Net Demand	Collection	Date	Status
Jul-24	31	₹533.42	₹3736.12	₹379.69	₹4649.23	₹10517	2024-11-23	Paid
Aug-24	21	₹431.67	₹2240.1	₹319.9	₹2991.67	₹0	-	Pending
Sep-24	19	₹379.64	₹2138.45	₹289.33	₹2807.42	₹0	-	Pending
Oct-24	10	₹364.92	₹1087.8	₹276.6	₹1729.32	₹8392	2024-02-17	Paid
Nov-24	30	₹305.1	₹3356.7	₹325.21	₹3987.01	₹0	-	Pending

Figure No 6.2.4 Billing Details

This page displays the billing details of a student, allowing the admin to view fee-related

information after searching by admission or registration number. It provides a student summary section with basic details and an overview of payments. Below that, a detailed table shows month-wise billing records, including charges such as electricity, mess, and service fees, along with the total demand, amount paid, payment date, and status (paid or pending). This structured view helps the admin easily track payments, identify pending dues, and manage hostel fee records efficiently.

REGISTRATIONS:

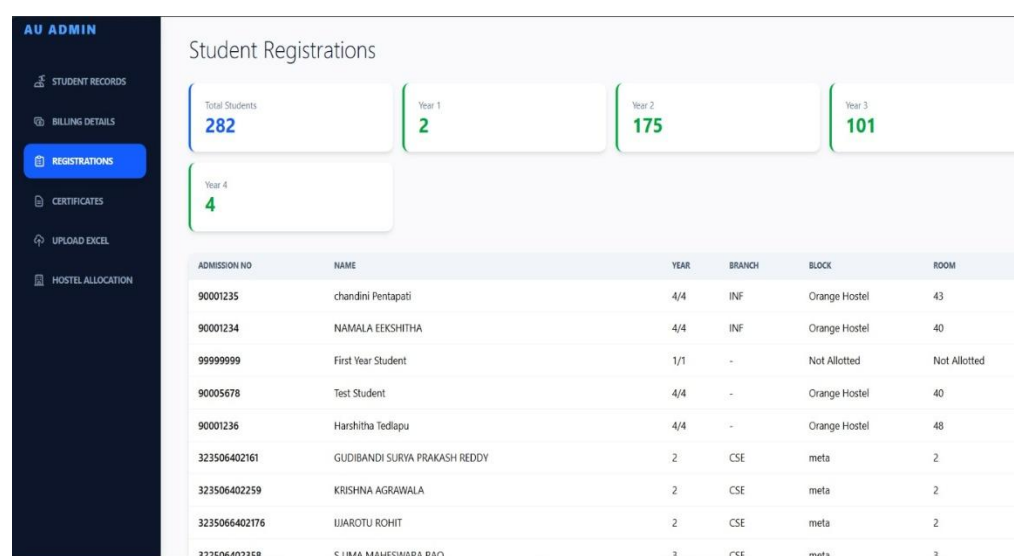


Figure No 6.2.5 Registrations

This page displays the student registrations managed within the system. At the top, it provides a summary of the total number of registered students along with a year-wise distribution, helping the admin quickly understand how many students belong to each academic year. Below the summary, a detailed table lists individual student records, including admission number, name, year, branch, assigned hostel block, and room number. It also clearly indicates cases where rooms are not yet allotted. This page helps the admin monitor registrations, track allocations, and manage student data efficiently in a structured manner.

CERTIFICATES:

The certificates dashboard serves as a centralized interface where the admin can generate official hostel-related documents for students. It provides options to create different types of certificates such as Resident Certificate, No Dues Certificate, and Fee Estimation Slip. Each option includes a “Generate PDF” feature, allowing the admin to quickly produce and download verified documents. This page simplifies administrative work by enabling instant

generation of formal records whenever required.

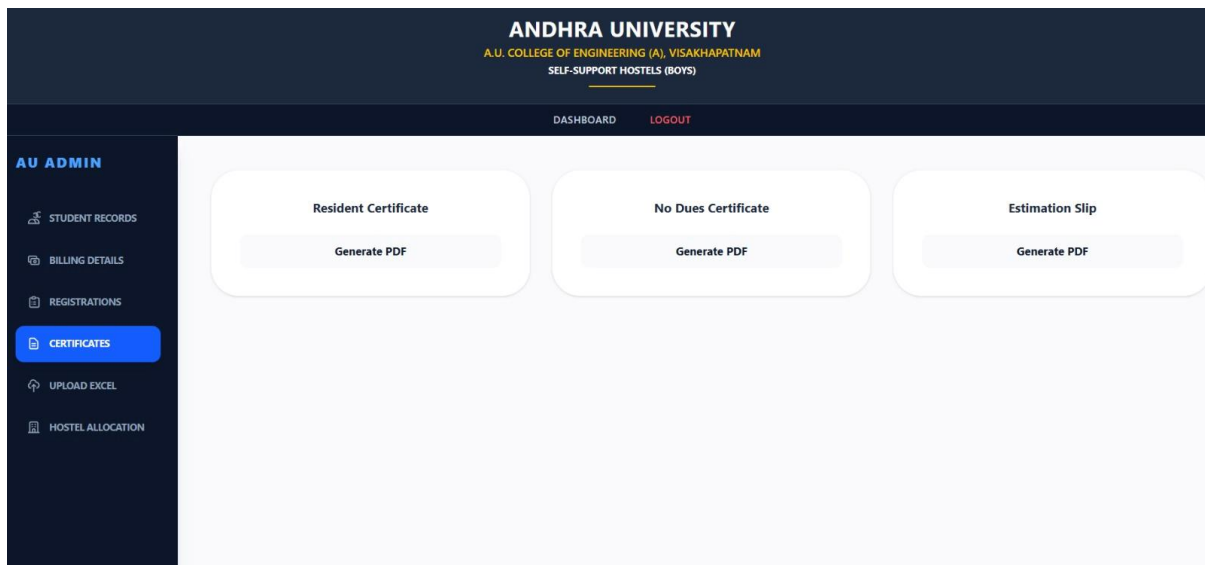


Figure No 6.2.6 Certificates

Resident certificate

The image shows a 'RESIDENT CERTIFICATE' form for Andhra University. At the top is the university's logo. Below it, the text reads 'ANDHRA UNIVERSITY', 'A.U. COLLEGE OF ENGINEERING (A), VISAKHAPATNAM', and 'SELF-SUPPORT HOSTELS (BOYS)'. The title 'RESIDENT CERTIFICATE' is underlined. The form contains the following text: 'This is to certify that Mr./Ms. NAMALA EEKSHITHA Reg. No 322107311030 Class 4/4 Branch INF degree B.Tech is a boarder of AU Engineering College Hostels. Hostel Block Orange Hostel Room No 40'. Below this is a section for 'Address of the Candidate:' with a text box containing '57-8-5 , Gokul Nagar , kancharapalem'. At the bottom, it states 'This certificate is issued to enable the candidate for applying for **passport** purposes only.' and includes a date 'Date: 28/4/2026' and a signature line for the 'CHIEF WARDEN'.

Figure No 6.2.6.1 Resident Certificate

From this dashboard, the Resident Certificate can be generated, which officially confirms that a student is residing in the hostel. It includes details such as the student's name, registration number, academic information, hostel block, and room number, along with their address. This certificate is typically used for official purposes like passport applications or address verification, making it an important document for students.

No Dues:



The image shows a 'NO DUES CERTIFICATE' from Andhra University, A.U. College of Engineering (A), Visakhapatnam, Self-Support Hostels (Boys). The certificate is for Mr. NAMALA EEKSHITHA, Reg. No. 322107311030, Class 4/4, degree B.Tech, Branch INF. It states that the student has cleared all hostel dues and no dues are pending against him as per hostel records. The date is 28/4/2026 and it is signed by the Chief Warden.

ANDHRA UNIVERSITY
A.U. COLLEGE OF ENGINEERING (A), VISAKHAPATNAM
SELF-SUPPORT HOSTELS (BOYS)

NO DUES CERTIFICATE

This is to certify that Mr. **NAMALA EEKSHITHA** Reg. No **322107311030**
Class **4/4** degree **B.Tech** Branch **INF**

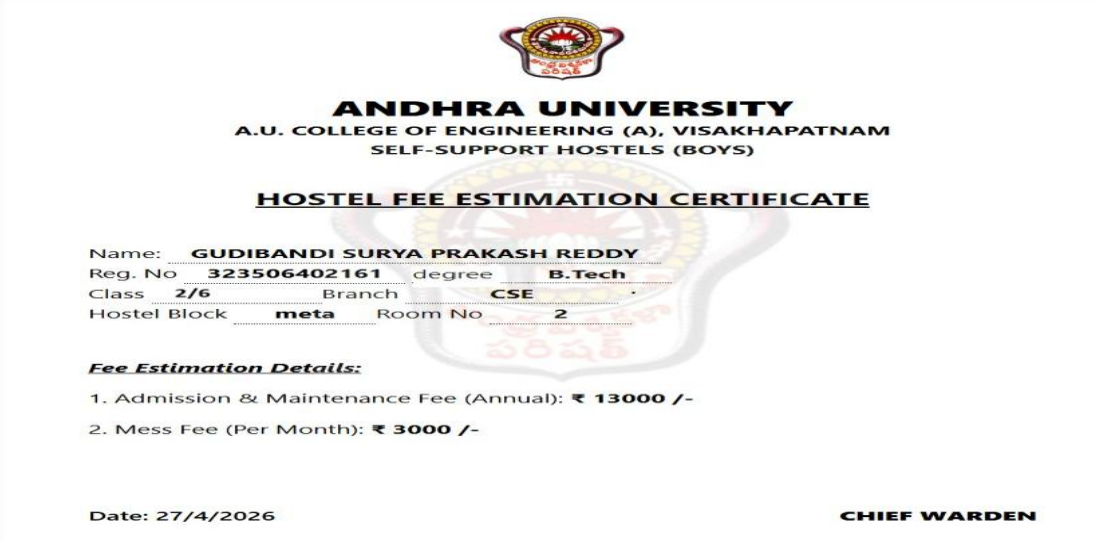
It is certified that the above student has cleared all hostel dues and no dues are pending against him as per hostel records.

Date: 28/4/2026 **CHIEF WARDEN**

Figure No 6.2.6.2 No Dues Certificate

Similarly, the No Dues Certificate is issued to confirm that a student has cleared all hostel-related payments and has no pending dues. It includes academic and identification details of the student along with a formal statement verifying that there are no outstanding charges. This certificate is especially important during course completion, hostel vacating, or other formal clearances.

Fee Estimation certificate:



The image shows a 'HOSTEL FEE ESTIMATION CERTIFICATE' from Andhra University, A.U. College of Engineering (A), Visakhapatnam, Self-Support Hostels (Boys). The certificate is for GUDIBANDI SURYA PRAKASH REDDY, Reg. No. 323506402161, degree B.Tech, Class 2/6, Branch CSE, Hostel Block meta, Room No 2. It provides a breakdown of fees: Admission & Maintenance Fee (Annual) ₹ 13000 /- and Mess Fee (Per Month) ₹ 3000 /-. The date is 27/4/2026 and it is signed by the Chief Warden.

ANDHRA UNIVERSITY
A.U. COLLEGE OF ENGINEERING (A), VISAKHAPATNAM
SELF-SUPPORT HOSTELS (BOYS)

HOSTEL FEE ESTIMATION CERTIFICATE

Name: **GUDIBANDI SURYA PRAKASH REDDY**
Reg. No **323506402161** degree **B.Tech**
Class **2/6** Branch **CSE**
Hostel Block **meta** Room No **2**

Fee Estimation Details:

1. Admission & Maintenance Fee (Annual): ₹ 13000 /-
2. Mess Fee (Per Month): ₹ 3000 /-

Date: 27/4/2026 **CHIEF WARDEN**

Figure No 6.2.6.2 Hostel Fee Estimation Certificate

The Fee Estimation Certificate provides a breakdown of the hostel fee structure, including admission and maintenance charges as well as monthly mess fees. It gives a clear estimate of the expenses a student is expected to incur, which is useful for financial planning or official submissions. Together, these certificate pages are interconnected through the dashboard, enabling the admin to efficiently generate, manage, and provide essential documents within the

hostel management system.

UPLOAD EXCEL DATA:

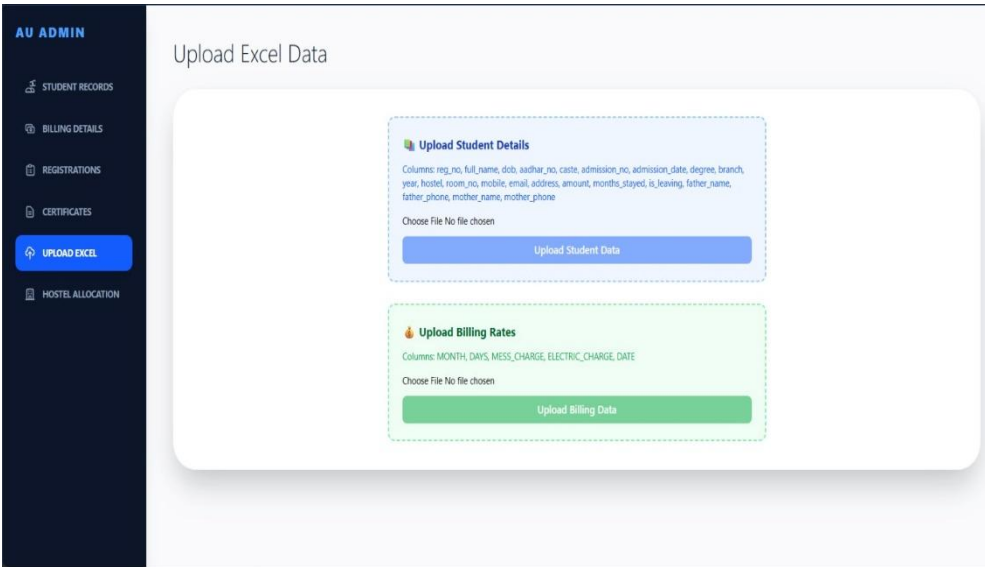


Figure No 6.2.7 Upload excel sheet

This page allows the admin to upload data into the system using Excel files, making bulk data management efficient and time-saving. It provides two main sections: one for uploading student details and another for uploading billing rates. The student upload section accepts structured data such as personal, academic, and hostel-related information, while the billing upload section is used to add fee-related data like monthly charges and service costs. Each section includes a file selection option and an upload button, ensuring a simple and guided process. This feature helps in quickly importing large datasets, reducing manual entry and improving accuracy in the system.

HOSTEL ALLOCATION:

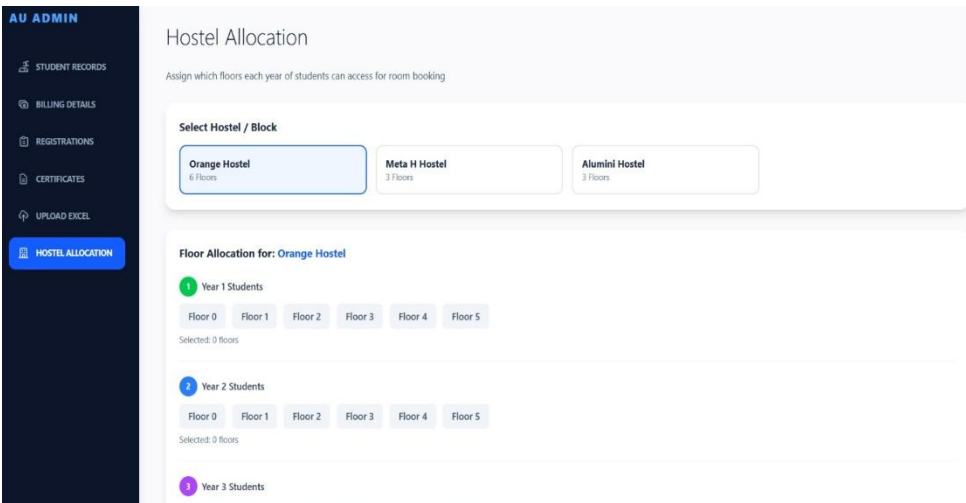
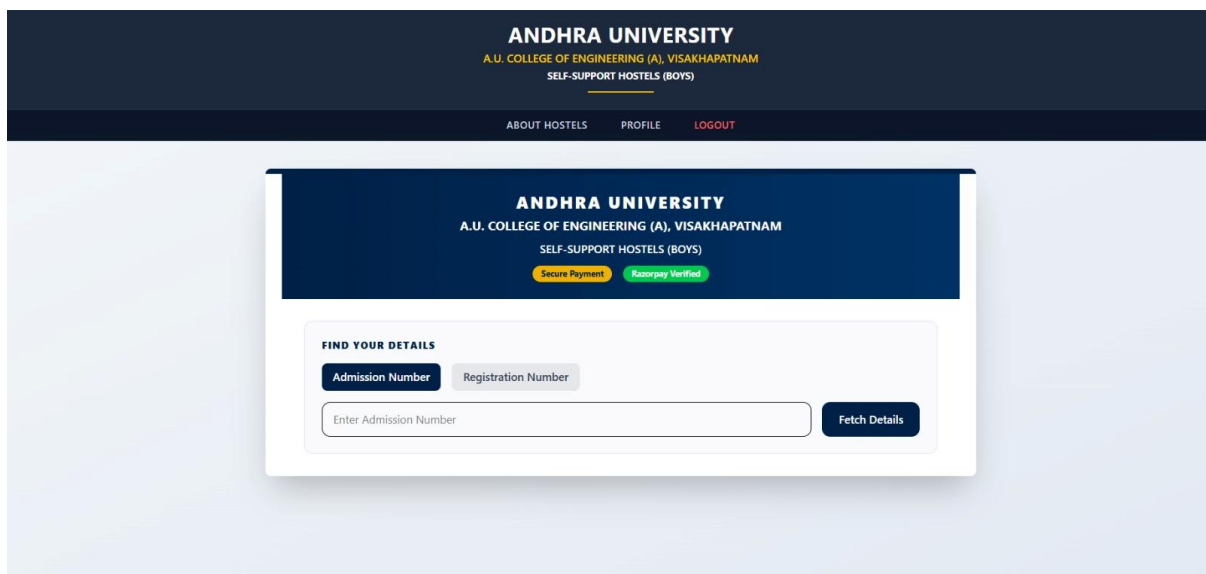


Figure No 6.2.8 Hostel Allocation

The Hostel Allocation page in the admin module is used to manage how hostel floors are assigned to students based on their academic year. The admin first selects a specific hostel or block (such as Orange Hostel, Meta H Hostel, or Alumni Hostel), each with a defined number of floors, and then configures which floors are accessible to different groups like first-year, second-year, and third-year students. For each year, multiple floor options are provided, allowing flexible and controlled allocation. This ensures that students can only view and book rooms on the floors assigned to their category, helping maintain proper organization, avoid overcrowding, and enforce structured accommodation within the hostel system.

KNOW YOUR DETAILS (mess fee payment):



The screenshot displays the 'Know Your Details' portal for Andhra University. The page has a dark blue header with the university name and navigation links. Below the header is a white card with a blue header and a search form. The search form has two tabs: 'Admission Number' and 'Registration Number'. A text input field is labeled 'Enter Admission Number' and a 'Fetch Details' button is next to it. Above the search form, there are two buttons: 'Secure Payment' and 'Razorpay Verified'.

Figure 6.2.9 Know Your Details

This page is the mess fee payment portal, designed to help students easily access and pay their hostel mess charges. It provides a simple interface where students can search for their details using either their admission number or registration number. Once the correct number is entered, the system fetches the student's billing information for further processing. The page emphasizes secure transactions, indicating that payments are verified and handled safely. Overall, it serves as a dedicated and user-friendly entry point for students to view their dues and proceed with mess fee payments efficiently.

Mess Fee Payment:

ANDHRA UNIVERSITY

COLLEGE OF ENGINEERING (A) HOSTEL

Official Mess Fee Payment Portal

Secure PaymentRazorpay Verified

1. STUDENT INFORMATION

FULL NAME *

GUDIBANDI SURYA PRAKASH REDDY

ROLL NUMBER *

323506402161

ROOM NUMBER

2

CLASS/YEAR *

2

2. PAYMENT DETAILS

SELECT MONTH *

Select Month

NUMBER OF DAYS *

Auto-filled from database

AMOUNT (INR) *

3000

PAYMENT DATE *

dd-mm-yyyy

Payment Summary

Student Name:

GUDIBANDI SURYA PRAKASH REDDY

2. PAYMENT DETAILS

SELECT MONTH *

Sep-24

NUMBER OF DAYS *

19

AMOUNT (INR) *

2807.42

PAYMENT DATE *

dd-mm-yyyy

Payment Summary

Student Name:

GUDIBANDI SURYA PRAKASH REDDY

Roll Number:

323506402161

Month:

Sep-24

Number of Days:

19 days

Total Amount:

₹2807.42/-

☐ Test Mode Payment (Skip Razorpay, mark as success)

Search Another Student

PROCEED TO PAY ₹2807.42

Figure No 6.2.9.1 mess fee payment

61

CONCLUSION AND FUTURE SCOPE

CONCLUSION AND FUTURE SCOPE

CONCLUSION:

The Digitalization of Hostel Management System has been successfully developed to automate and simplify hostel operations through a centralized web-based platform. The system addresses the limitations of manual processes by providing efficient solutions for student registration, profile management, room allocation, fee payment, and certificate generation.

The implementation using React, Django REST Framework, and PostgreSQL ensures a scalable, secure, and reliable system. Features such as real-time room allocation, secure online payment integration, and automated notifications enhance system efficiency and user experience. Role-based access control further ensures proper management and data security.

The system effectively reduces manual effort, minimizes errors, and improves transparency in hostel administration. It provides a structured and user-friendly interface for students, wardens, and administrators, making hostel management more organized and efficient. Overall, the project demonstrates a practical application of modern web technologies in solving real-world problems and provides a strong foundation for future enhancements and scalability.

FUTURE SCOPE:

The Digitalization of Hostel Management System has strong potential for future enhancements to meet evolving institutional needs. Deploying the system on the cloud can enable remote access, scalability, and reliable data storage. Enhanced role-based access can further improve control and usability for administrators and wardens.

Features such as mobile application support, real-time notifications, and intelligent room allocation can improve user experience and resource utilization. Advanced analytics can help monitor occupancy, payments, and overall hostel operations, supporting better decision-making. Integration with institutional ERP systems can ensure seamless data flow.

Additional functionalities like complaint management, visitor tracking, and online leave systems can enhance efficiency and student convenience. Overall, these improvements can transform the system into a smart, scalable, and fully automated hostel management solution.

Furthermore, incorporating AI-based insights and chatbot support can enable faster query resolution and smarter decision-making. Enhanced security features and continuous system updates will ensure reliability, adaptability, and long-term sustainability of the system.

REFERENCES

REFERENCES

- 1) Pressman, Roger S., and Bruce R. Maxim. *Software Engineering: A Practitioner's Approach*. 8th ed., McGraw-Hill Education, 2014.
(A comprehensive guide covering software development methodologies, system design, and project management principles.)
- 2) Silberschatz, Abraham, Henry F. Korth, and S. Sudarshan. *Database System Concepts*. 7th ed., McGraw-Hill Education, 2019.
(Provides detailed concepts on database design, relational models, normalization, and transaction management.)
- 3) Banks, Alex, and Eve Porcello. *Learning React: Modern Patterns for Developing React Apps*. 2nd ed., O'Reilly Media, 2020.
(Explains modern React development, component-based architecture, and frontend design practices.)
- 4) Microsoft. *Visual Studio Code Documentation*.
Available at: <https://code.visualstudio.com>
(Used as the primary development environment for coding and debugging.)
- 5) React Documentation. *React – A JavaScript Library for Building User Interfaces*. Meta Platforms, Inc.
Available at: <https://react.dev>
(Official documentation used for building dynamic and responsive frontend components.)
- 6) Django Software Foundation. *Django Web Framework Documentation*.
Available at: <https://www.djangoproject.com>
(Provides guidelines for backend development, authentication, and database integration.)
- 7) Django REST Framework. *Django REST Framework Documentation*.
Available at: <https://www.django-rest-framework.org>
(Used for designing RESTful APIs and handling client-server communication.)
- 8) PostgreSQL Global Development Group. *PostgreSQL Documentation*.
Available at: <https://www.postgresql.org>
(Reference for relational database management, query optimization, and data integrity.)
- 9) Razor pay Software Pvt Ltd. *Razor pay Payment Gateway Documentation*.
Available at: <https://razorpay.com/docs>
(Used for integrating secure online payment processing into the system.)
- 10) Postman Inc. *Postman API Platform Documentation*.
Available at: <https://www.postman.com>
(Provides tools and documentation for testing and validating APIs.)